

The stack, explained

Every part of ReelWalk in plain English: what it does, how the pieces talk, why each was chosen, what it costs, and what is real today versus only designed. Learn it in English first, then say it in Japanese.

How to use this guide.

1. Read the big picture and the four request flows until you can draw the system map from memory.
2. Learn each decision's one-line answer, then its "why" and "rejected" lists: interviewers dig for trade-offs.
3. Use the XML briefing with a voice agent and let it grill you section by section.
4. When your English answers are solid, practise the Japanese versions on the study site.

Generated 2026-10-07 from the ReelWalk repository and its architecture decision records.

Contents

1. Your pitch: 30 seconds, 90 seconds, 3 minutes
2. The big picture: system map and code layout
3. Four request flows, step by step
4. Reliability: job states, duplicate deliveries, crashes
5. Platform and delivery: Kubernetes, Helm, CI/CD and GitOps
6. Architecture decisions and their trade-offs
7. Every component (29 topics)
8. What's real, what's designed
9. General interview questions

1. Your pitch

30 seconds

ReelWalk is a phone-first editor for making real-estate Reels. You arrange photos, videos and 360 photos, add text and music, and export. What makes it different is a "you are here" marker that moves across the floor plan. Exports go through a queue and are rendered on a separate worker. I built it with technology close to yours: Next.js, Prisma, Kubernetes and so on.

90 seconds

ReelWalk is an app for making property-listing Reels on a phone. The goal is that agents can make the video themselves instead of outsourcing it. You can cut, reorder, add text and music, and animate 360 photos. It has two parts: a Next.js web app and a worker that renders the video. Pages are Server Components, and every change goes through a Server Action. Data lives in PostgreSQL through Prisma; videos and other files go to S3. The browser uploads media straight to S3 with a presigned URL. When you press Export, the job is saved and only its ID is sent to SQS. SQS can deliver the same message twice. So I made the worker idempotent. Status changes are conditional updates, so a job can never complete twice. Locally it runs with Helm on kind; production is deployed to AWS with Terraform. GitHub Actions runs the tests, vulnerability scans and end-to-end tests. Production runs on App Runner and Fargate tasks that only run when needed, for ten-something dollars a month.

3 minutes, with one hard problem

Let me explain ReelWalk in a bit more detail. Property videos are expensive to outsource and slow to edit yourself. So I made an app that does the whole Reel on a phone. Its signature feature is a floor-plan marker that moves in step with the video. Architecturally, the Next.js web app and the rendering worker are separate. Rendering uses a lot of CPU and memory and can take minutes. That keeps the web app light and lets me scale only the workers. The preview and the export are drawn by the same React component. So what you see on the phone matches the exported video. Now let me talk about the hardest part. It was making sure each export job completes exactly once. At first I passed jobs through a Redis list. But when a worker crashed mid-render, the job simply disappeared. So I switched to SQS. SQS hides a received message from other consumers for a set time. If it isn't deleted in time, it's delivered to another worker. The catch is that the same message may arrive twice. So I made the worker idempotent: two deliveries still give one result. First, status updates are conditional. "Mark done only if still running" means the late one changes nothing. Second, the output file is named after the job ID. Rendering twice just overwrites the same file. Third, a heartbeat: every 20 seconds the worker writes to the database that it's alive. If it stops for 60 seconds, another worker takes over. A manual retry bumps a generation number, so old messages are discarded. After three failures the message goes to a dead-letter queue and the job is marked failed. I made that decision logic a pure function and tested each case with Vitest. There was one more problem that I fixed later. At first, saving the job and sending the message weren't one transaction. If the process died right after the save, a job was left with no message. So I added a transactional outbox and closed that gap. The message is written to the database together with the job, and sent to the queue afterwards. This taught me to design on the assumption that things will fail.

When they ask about AWS

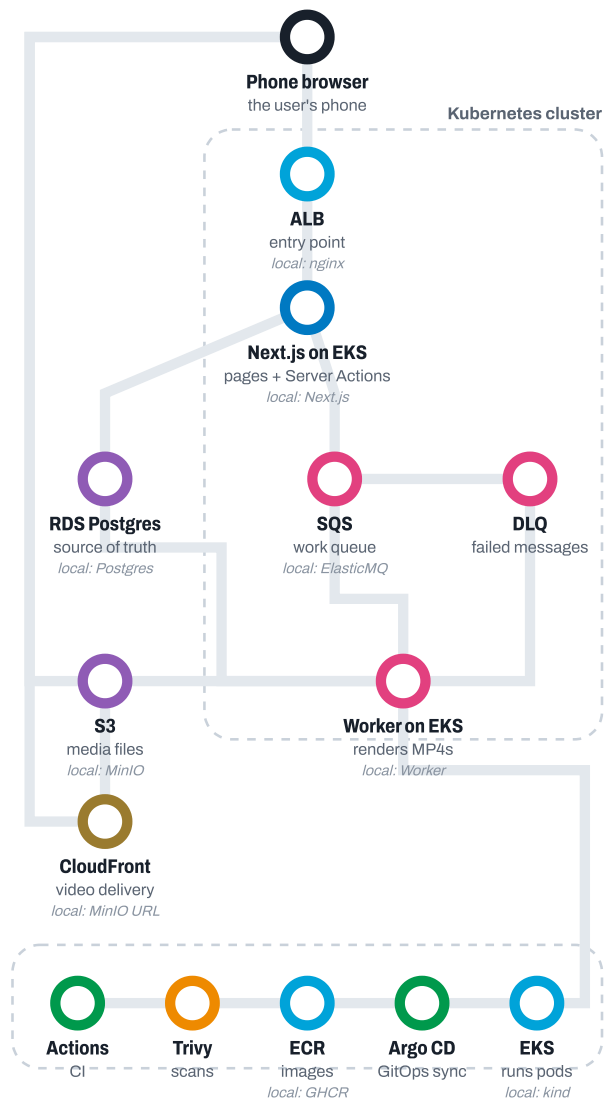
Production runs on AWS. The infrastructure is managed with Terraform: S3, CloudFront, SQS, ECR and IAM are all code. At first I stood production up on EKS and RDS, and ran it for a day. But it came to about \$250 a month, and most of that was idle cost. So I moved the same images to App Runner, and to Fargate tasks that start only when needed. When an export is queued, the web app starts one Fargate task, and it stops by itself when it is done. The monthly cost dropped to ten-something dollars. The database is on Supabase's free tier, the only piece outside AWS. Kubernetes is still used for verification, with Helm on a local kind cluster. That decision is recorded in an ADR with the reasons and the numbers. I make a point of being clear about what runs and what I retired. After joining, I want to learn how AWS is run at a larger scale.

Why this stack fits the job

The stack in your job post is almost the same as ReelWalk's. I matched it on purpose and built it understanding why each piece is there. Server Components read data directly through Prisma. Server Actions validate input with zod and check the workspace. Turborepo makes sure the Prisma client is generated before anything else. Logic is tested with Vitest, and user flows with Playwright. In CI, Trivy scans the dependencies and the images. I removed package managers and dev dependencies from the runtime images. The Helm chart runs as non-root and sets resource limits. For Argo CD, I installed it on local kind and confirmed that a merge to main deploys automatically. Production on AWS is small, but I went as far as changing the architecture after looking at the cost. So I'm clear about where I can contribute right away and what I still need to learn.

2. The big picture

A phone-first editor for Instagram Reels of property listings. The web app (Next.js) never renders video: it saves an export as a job in Postgres and puts the job's ID on a queue. Separate workers take jobs from the queue, render the MP4 with Remotion and store it in S3. Uploads go straight from the phone to S3.



Read the map in one breath

Phone → entry point → Next.js. Next.js reads and writes Postgres and sends jobs to SQS. Workers pull from SQS, read media from S3, render, write the MP4 back to S3 and the status to Postgres. Messages that fail three times go to the dead-letter queue. In production CloudFront delivers the finished videos.

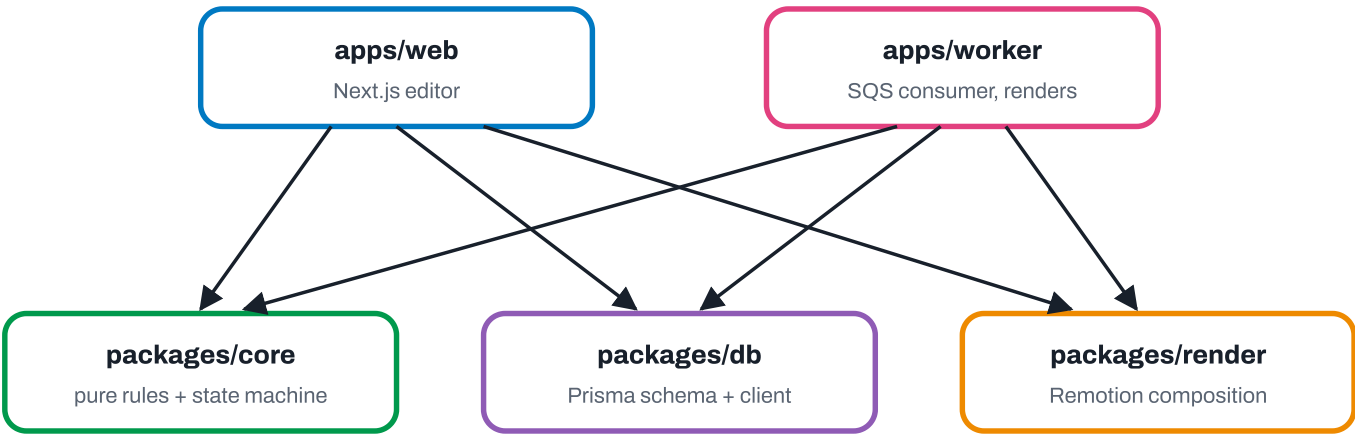
Local stand-ins (same code, different endpoints)

In production (AWS)	Runs locally as
ALB	nginx
Next.js on EKS	Next.js
RDS Postgres	Postgres
SQS	ElasticMQ
S3	MinIO
Worker on EKS	Worker
CloudFront	MinIO URL
ECR	GHCR
EKS	kind

The bottom row is delivery

GitHub Actions tests and scans every change; on main it pushes images to GHCR and records the tag in Git; Argo CD syncs the cluster to Git.

Code layout: one repo, shared packages



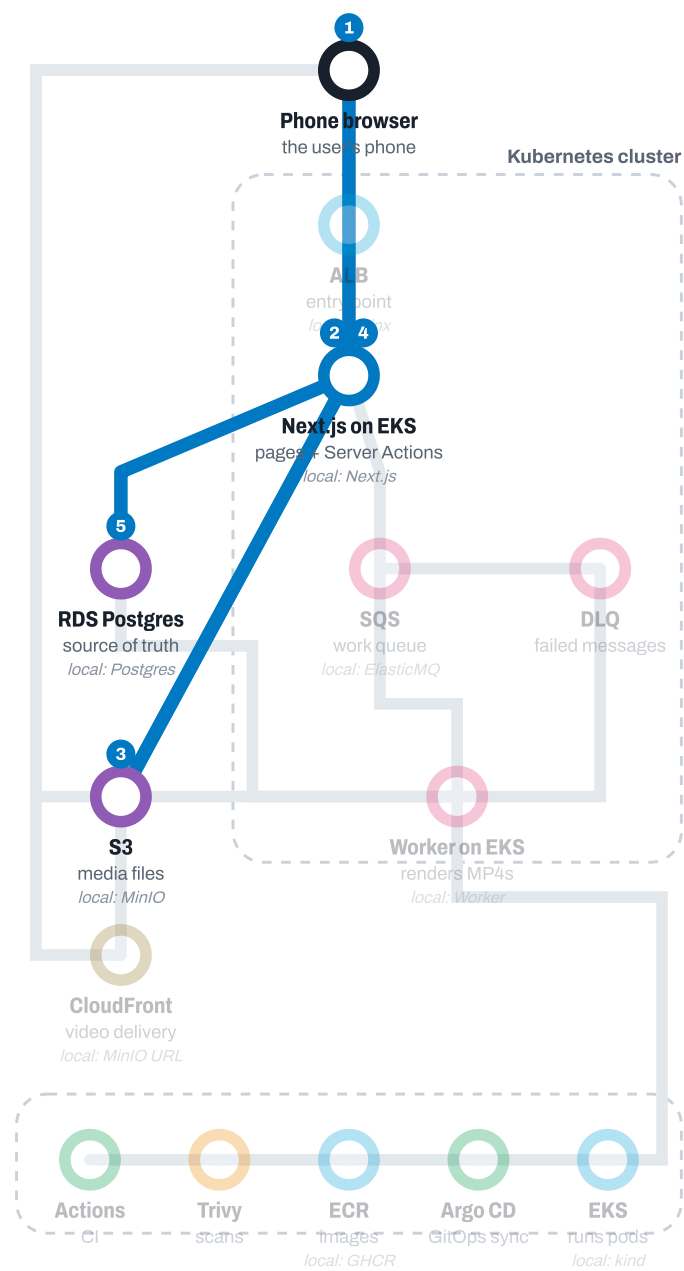
Web and worker share types and rules through packages, not through HTTP. `packages/core` holds pure, unit-tested logic: timeline edits, Instagram rules, the job state machine, retry and delivery decisions. Turborepo runs typecheck, test and build in dependency order, generating the Prisma client first.

3. Four request flows

Each diagram numbers the stops in order. Being able to walk an interviewer through these is the single most useful thing to practise.

Uploading media

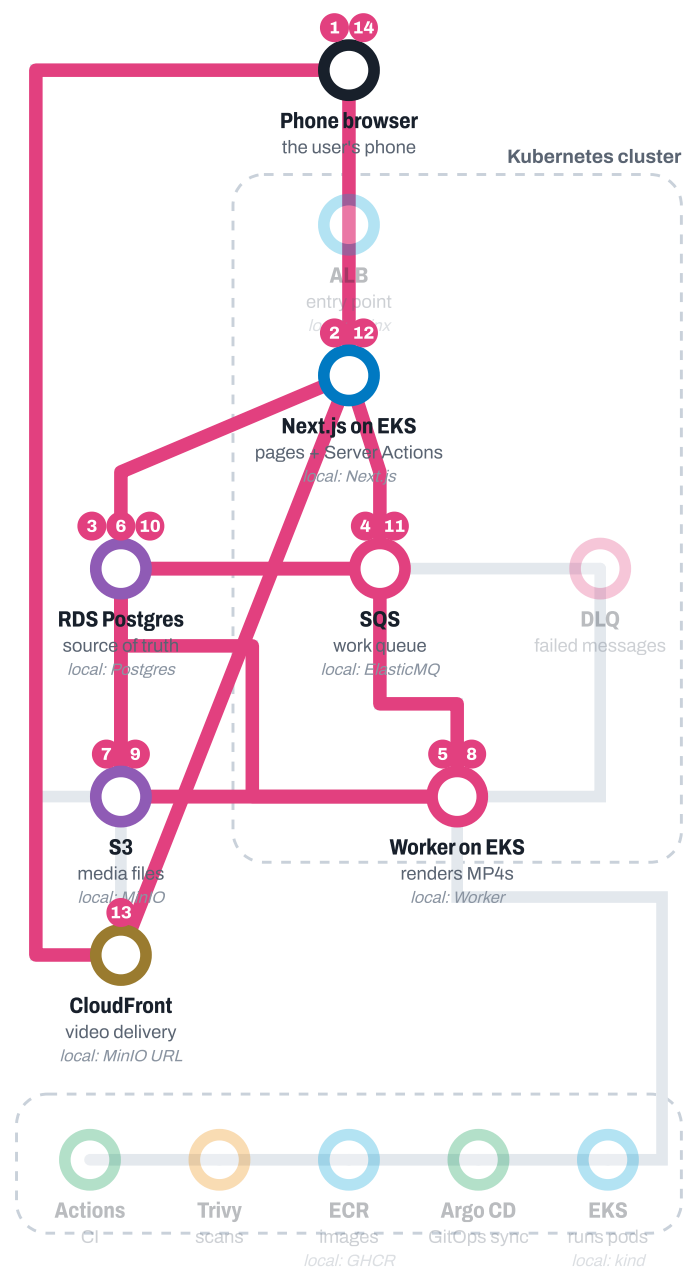
How a 360 photo or clip gets from the phone into storage without passing through the web server.



1. **Phone browser.** The user picks videos or 360° photos on their phone.
2. **Next.js on EKS.** A Server Action issues a presigned URL. Before that, it validates the file type and size.
3. **S3.** The browser uploads straight to S3. Large files never pass through Next.js, so the server stays light. Locally, MinIO stands in for S3.
4. **Next.js on EKS.** When the upload finishes, the server checks the file.
5. **RDS Postgres.** Prisma saves a record for the media asset. The file itself lives in S3; the information about it lives in the database.

Export and render

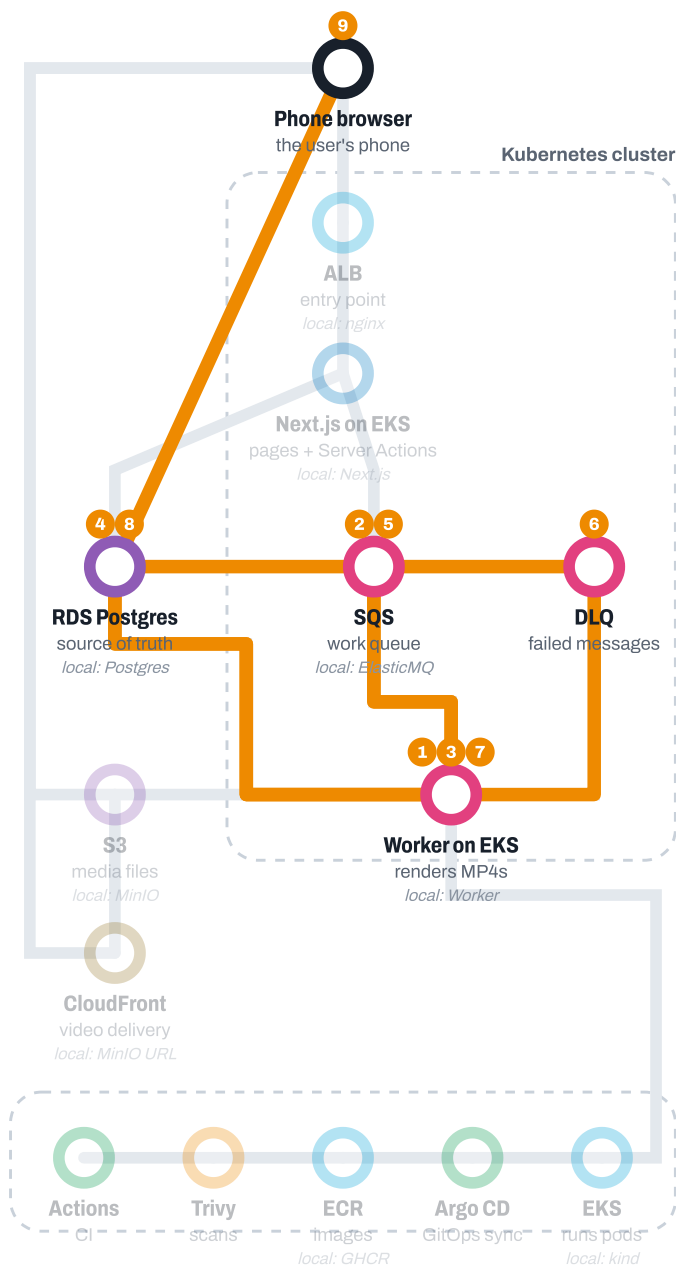
One tap on Export, followed through the queue, the worker and back to the phone.



1. **Phone browser.** The user taps Export in the editor.
2. **Next.js on EKS.** A Server Action checks the input and the workspace.
3. **RDS Postgres.** It saves a render job in the QUEUED state.
4. **SQS.** It sends SQS a small message with just the job ID. All the details stay in the database. So the message can never disagree with the data.
5. **Worker on EKS.** A worker receives the message by long polling.
6. **RDS Postgres.** A conditional update moves the job to RUNNING. The update only succeeds when the status and generation match. So two workers never take the same job.
7. **S3.** It downloads the media from S3.
8. **Worker on EKS.** Remotion renders the MP4 with the same component as the preview. It regularly writes progress and a heartbeat.
9. **S3.** It uploads the finished MP4 to S3.
10. **RDS Postgres.** It marks the job SUCCEEDED and records the output.
11. **SQS.** Only then does it delete the SQS message. Saving before deleting means a crash midway never loses the work.
12. **Next.js on EKS.** The screen reads progress from the database and shows it.
13. **CloudFront.** On AWS, the design delivers the video through CloudFront.
14. **Phone browser.** The user can play, save and share the video.

When a worker crashes

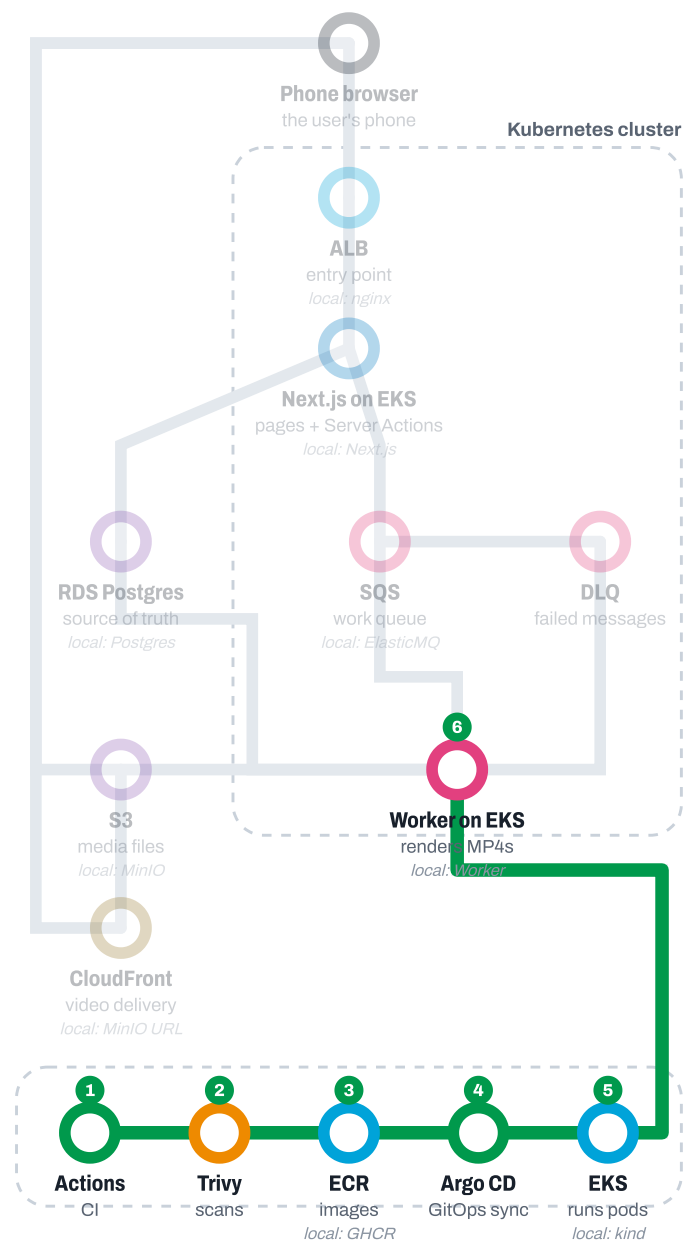
At-least-once delivery, heartbeats and the dead-letter queue, step by step.



1. **Worker on EKS.** The worker pod crashes in the middle of a render.
2. **SQS.** The message was never deleted, so it is still in SQS. Once the worker stops, it stops extending the visibility timeout. When the timeout runs out, the message becomes visible again.
3. **Worker on EKS.** Another worker receives the same message.
4. **RDS Postgres.** First it checks the job's state in the database. Messages from an old generation, or for a finished job, are thrown away. If the heartbeat is fresh, it leaves the job to the other worker. If the heartbeat is stale, it decides the worker died and takes over.
5. **SQS.** If a render throws, it retries with exponential backoff. It creates the wait by extending the visibility timeout.
6. **DLQ.** After three receives without success, the message moves to the dead-letter queue.
7. **Worker on EKS.** The worker also reads the DLQ and fails any job left behind. If every attempt crashed its pod, nobody wrote FAILED yet.
8. **RDS Postgres.** The job becomes FAILED and the error is kept.
9. **Phone browser.** The user can retry manually from the screen. A retry bumps the generation, so old messages get ignored.

From code to production

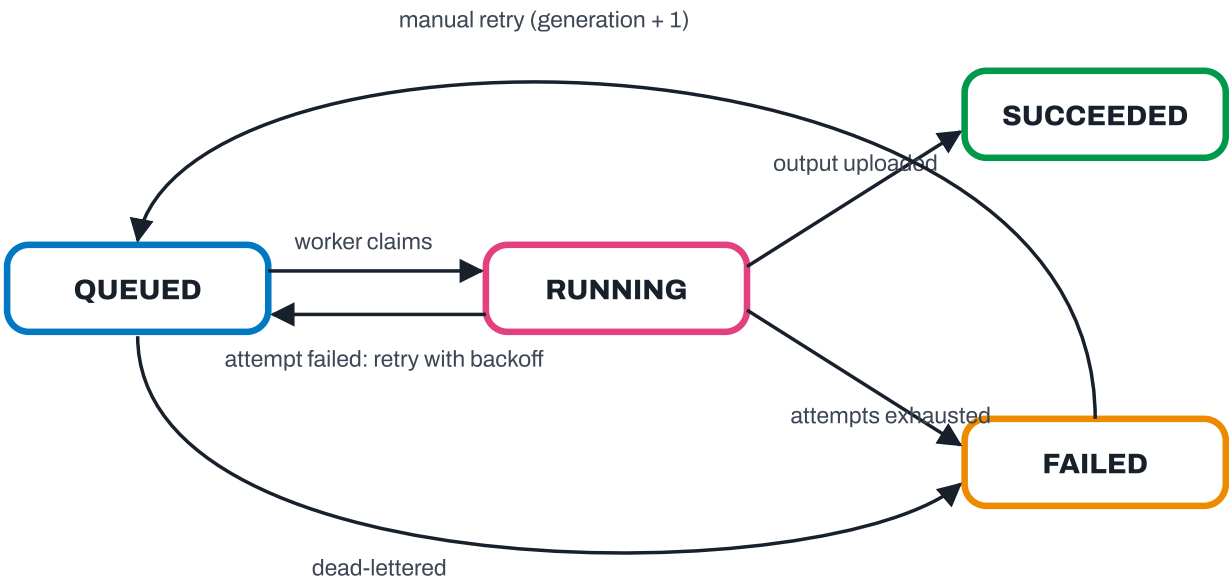
The CI pipeline that runs today, the GitOps loop on kind, and how production on AWS is deployed (ADR 0015).



1. **Actions.** A push starts GitHub Actions. It runs lint, type checks, unit tests and a Helm lint.
2. **Trivy.** Trivy scans the dependencies, the config and the images. A serious vulnerability stops the pipeline.
3. **ECR.** It builds the Docker images and pushes them to GHCR, GitHub's container registry. An ECR push is wired up too, but it hasn't run yet: the AWS account is waiting on verification.
4. **Argo CD.** Argo CD syncs the cluster with the Helm chart in Git. On local kind, I confirmed that a merge to main is rolled out automatically.
5. **EKS.** Kubernetes swaps the pods with a rolling update. Traffic only goes to a pod once its readiness probe passes. Locally, I checked this flow on a kind cluster.
6. **Worker on EKS.** Web and worker scale separately.

4. Reliability

Render job states



The legal transitions are a pure function (`packages/core/src/job-status.ts`). Every change is a single conditional update: the `where` clause includes the expected status and generation, so if another writer got there first, zero rows change.

What a worker does with each delivery

Job row when the message arrives	Action
missing, SUCCEEDED, FAILED, or a different generation	delete the message (duplicate or stale)
QUEUED	claim and render
RUNNING, heartbeat fresh	leave it; another worker has it
RUNNING, heartbeat older than 60 s	claim and render; the previous worker died

Timing: visibility timeout 120 s, heartbeat every 20 s, stale after 60 s, 3 attempts, backoff 15 s then 30 s.

Idempotency in four facts.

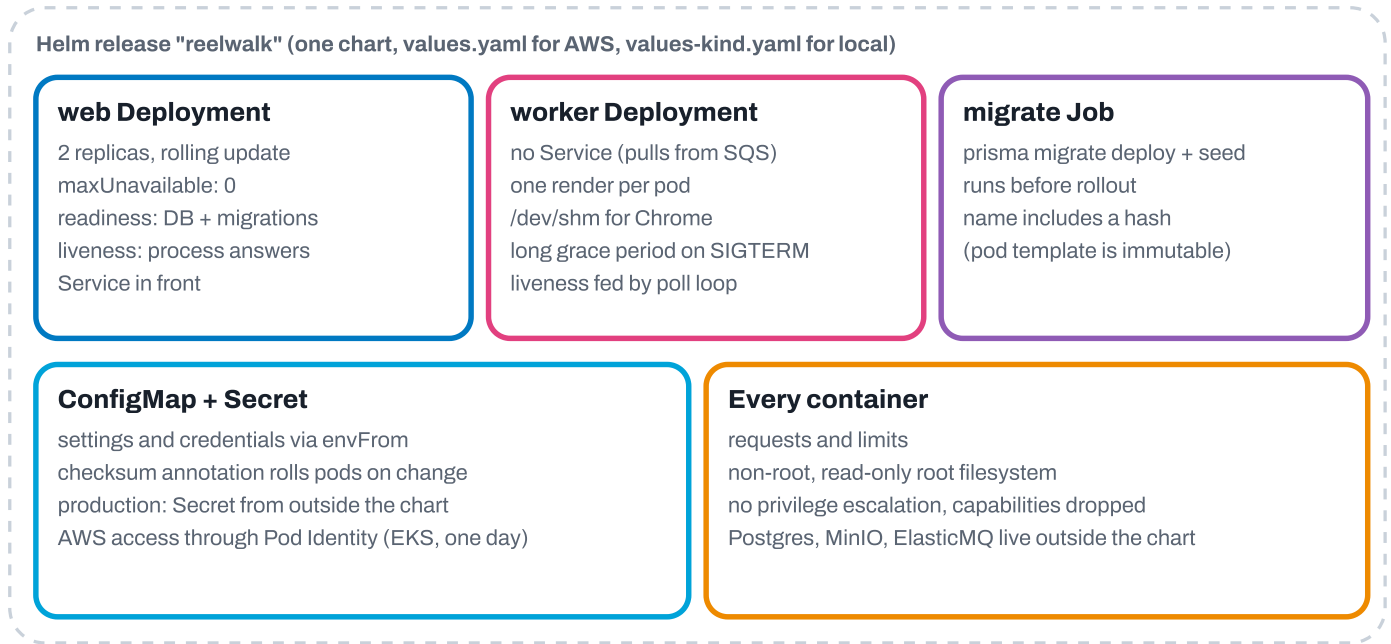
1. The output key is `renders/<jobId>.mp4` , so a second render overwrites the same file.
2. `RenderOutput.jobId` is unique and written with an upsert: one output per job.
3. Finished jobs and old generations are discarded without rendering.
4. A manual retry bumps the generation, so late messages from the old run are ignored.

Transactional outbox. The job row and its queue message are written in one transaction (the `OutboxMessage` table). The web app sends the message right after the commit, and a relay loop in every worker sends whatever is left, so a saved job always gets its message (ADR 0010).

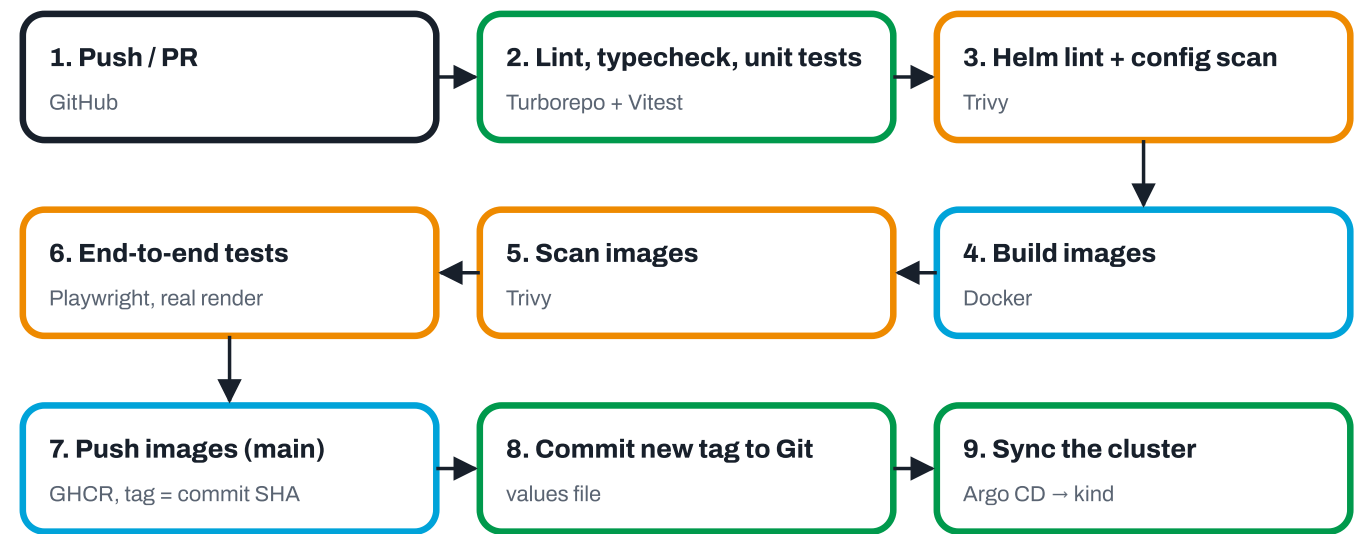
Honest gap: there is no per-claim fencing token yet.

5. Platform and delivery

What the Helm chart creates



From a commit to a running pod



CI never touches the cluster. It records the new image tag in Git; Argo CD notices the commit and rolls it out, so a merge to main is a deploy and a revert is a rollback. That GitOps loop runs on the local kind cluster. On main, CI also pushes the same images to ECR through a GitHub OIDC role (no stored keys), and production pulls from there.

5b. Production on AWS, and the cost decision

Learn this page cold. It is the most likely follow-up question and the best story in the project: you built it the expensive way, measured it, and rebuilt it the cheap way, with the records to prove both.

What runs today (ADR 0015), about \$10-15 a month

Piece	Runs on	Notes
Web app	AWS App Runner, 0.5 vCPU / 1 GB, from the ECR image	HTTPS, custom domain and certificate are part of the service; no load balancer or ingress. Idle cost about \$5.
Render workers	ECS Fargate tasks started on demand	When an export is queued the web app calls ecs:RunTask; the task drains the queue and exits after four idle minutes. A few cents per render, nothing in between.
Queue	SQS + dead-letter queue	Unchanged: lease, heartbeat, retries, outbox.
Media	S3 + CloudFront	Presigned uploads from the phone; CloudFront serves finished videos.
Images	ECR, pushed by GitHub Actions via OIDC	Tag = commit SHA, immutable, scanned on push.
Config and secrets	SSM Parameter Store; IAM task and instance roles	No access keys anywhere in the runtime.
Database	Free Postgres tier outside AWS (Supabase)	The one non-AWS piece; chosen because RDS alone is \$15 a month.
Everything	Terraform, state in S3	Existing bucket and CloudFront were imported, not recreated.

What ran for one day (ADR 0014), about \$250 a month

EKS 1.34 with one t3.xlarge node, RDS Postgres 16, ingress-nginx behind a network load balancer, cert-manager with Let's Encrypt, Pod Identity for AWS access. It worked: a real render was verified end to end at reelwalking.com. The bill was \$73 control plane + \$120 node + \$17 load balancer + \$15 database, nearly all of it idle. It was destroyed the same evening.

How to tell it

- "I did run it on EKS. It proved the Helm chart, IAM, TLS and DNS on a real cluster. Then I looked at the bill and the workload and moved to App Runner plus on-demand Fargate, same images, same queue, about twenty times cheaper."
- Why not Remotion Lambda? Near-zero idle cost and the real long-term answer, but it replaces the worker and needs the WebGL transitions tested under Lambda. Deliberately a separate project.
- What is the trade-off? A cold start of one to two minutes for the first render after a quiet spell, and a small race where a worker exits as a message arrives (the next export picks it up).
- Kubernetes is not gone: the chart and Argo CD run on kind, and the EKS Terraform is in git history, 20 minutes from a rebuild.

6. Architecture decisions

Each decision record answers: what was the problem, what did you choose, why, what did you reject, and what does it cost. Learn the one-liner first.

ADR 0001: SQS for the render queue

"I used SQS because renders are long and can crash, and SQS gives me a lease, retries and a dead-letter queue for free; the cost is at-least-once delivery, so the worker is idempotent."

Problem. A render takes seconds to minutes, uses a lot of CPU and memory, and can crash its process. It can't run inside a web request. The first version used a Redis list with LPOP, which lost the job whenever a worker died mid-render and had no retries or dead-letter handling.

Decision

- Use SQS (ElasticMQ locally, same API) with a dead-letter queue.
- The message carries only { jobId, generation }. Postgres is the source of truth for everything else.
- While rendering, the worker extends the message's visibility timeout and writes a heartbeat to the job row. If the worker dies, both lapse and the message is delivered again.
- A failed attempt re-queues the job with exponential backoff by setting the visibility timeout. After 3 receives, SQS moves the message to the dead-letter queue.
- A small consumer on the dead-letter queue marks the job FAILED if it is still QUEUED or RUNNING, covering workers that crashed before recording the failure.

Why

- SQS gives a lease (visibility timeout), a receive count and a dead-letter queue out of the box.
- The tiny message can never disagree with the database.

Rejected

- **Redis list:** No visibility timeout, receive count or dead-letter queue; all would be hand-built.
- **Postgres as a queue (SKIP LOCKED):** Works at this scale, but long renders hold a transaction or need a hand-written lease. SQS already is that lease.
- **One Kubernetes Job per render:** Good isolation, but slow start per job and the API server becomes the queue.

Costs and consequences

- At-least-once delivery, so the worker must be idempotent (ADR 0003).
- Max attempts in the worker and maxReceiveCount on the queue must match, and are set in two places.
- The job row and the message live in two systems; they are kept consistent with a transactional outbox (ADR 0010).

ADR 0002: PostgreSQL with Prisma

“Postgres is the source of truth because the data is relational, and every status change is one conditional update, so two writers can never both win.”

Problem. The data is relational (workspace, media, reels, render jobs, outputs) and job status changes must be safe when two workers, or a worker and a user, act on the same job at once.

Decision

- PostgreSQL, accessed through Prisma from both the web app and the worker; schema, migrations and client live in packages/db.
- Status changes are single conditional updates, e.g. `updateMany where { id, generation, status: RUNNING }`. If someone else got there first, zero rows change. No read-then-write, no explicit locks.
- Legal transitions are a pure function in packages/core, tested with Vitest; the conditional updates mirror it.
- Migrations are committed SQL, applied by a one-shot Kubernetes Job (prisma migrate deploy) before the app rolls out.

Why

- Relations and constraints fit the data.
- Generated types are shared by the web app and the worker.

Rejected

- **DynamoDB:** No joins, and access patterns aren't stable enough yet to design keys around.
- **Raw SQL (asyncpg, the first version):** Fine, but no generated types shared between web and worker.

Costs and consequences

- Prisma 7 generates its client into the repo, so prisma generate must run before typecheck, test and build; Turborepo encodes that.
- Production would use RDS: only DATABASE_URL changes. Not yet deployed.

ADR 0003: Separate, idempotent render workers

“The worker is separate because rendering scales and fails differently from the web app, and it's idempotent because the queue can deliver a job twice.”

Problem. The web app is small and I/O bound; a render runs headless Chrome and ffmpeg and needs gigabytes of memory. They scale, fail and package differently (about 230 MB vs 1.7 GB images).

Decision

- Two deployables from one repo: apps/web (writes the job and sends the message) and apps/worker (receives and renders). They share code through packages, not HTTP.
- Output key is renders/<jobId>.mp4, so a second render overwrites the same object.
- RenderOutput.jobId is unique and written with an upsert.
- A delivery for a SUCCEEDED or FAILED job is deleted without rendering; one for a job with a fresh heartbeat is left alone.
- A manual retry increments generation, so messages from an older generation are discarded.

Why

- A web deploy never interrupts a render, and an out-of-memory render never takes the site down.
- One render per pod keeps memory predictable; throughput scales by adding pods.

Rejected

- **Rendering inside the web app:** Long, memory-hungry work would block requests and crash the site with the render.

Costs and consequences

- On SIGTERM the worker stops polling and finishes its job; if it's killed first, the job is picked up again after the visibility timeout.
- No per-claim fencing token yet: a worker wrongly judged dead could still write SUCCEEDED, which is harmless here because the output is identical.

ADR 0004: Kubernetes with a Helm chart

“Kubernetes gives me rolling updates, probes and separate scaling for web and worker, and Helm lets the same chart run on kind locally and, as it turned out, on EKS with one values file.”

Problem. Three workloads: a web app that must stay up during deploys, workers whose number should follow the queue, and a migration that runs once per release. Docker Compose has no rollout, health-based restarts or scaling.

Decision

- web: Deployment with 2 replicas, rolling update with maxUnavailable 0, and a Service. Readiness checks the database and migrations; liveness only checks the process answers.
- worker: Deployment with no Service, liveness fed by the poll loop, memory-backed /dev/shm for Chrome, long termination grace period.
- migrate: a Job named with a hash of image tag and chart version, because a Job's pod template is immutable.
- ConfigMap and Secret via envFrom, with a checksum annotation that rolls pods when either changes.
- Requests and limits, non-root, read-only root filesystem, no privilege escalation, all capabilities dropped.

Why

- The same manifests run on a laptop (kind) and would run on EKS.
- Helm values make the local and AWS differences explicit, one file each.

Rejected

- **ECS/Fargate:** Less to operate and reasonable at this size, but not portable to a local cluster.
- **Plain manifests or Kustomize:** Fine for one environment; values files describe the differences more clearly.

Costs and consequences

- Readiness vs liveness split: a database outage takes pods out of rotation instead of restarting them.
- Worker autoscaling is CPU-based and off by default; KEDA on queue depth is the planned improvement.
- In production the Secret comes from outside the chart; on EKS pods got AWS access through Pod Identity, which ran for one day (ADR 0014).

ADR 0005: kind before EKS

"I verified everything on a local kind cluster first; the same chart later ran on EKS for a day, which proved the AWS-only pieces, and kind stays the free daily demo."

Problem. EKS costs money from the first hour and needs VPC, IAM and add-ons before the first pod starts; the AWS account is also not available yet.

Decision

- Run the chart on kind (Kubernetes in Docker). A script builds images, loads them, deploys local stand-ins for RDS, S3 and SQS, and installs the release.
- Everything that differs from AWS is in values-kind.yaml. The app has no local mode: it always uses the S3 and SQS APIs through the AWS SDK.

Why

- The whole stack can be created and destroyed in minutes for free.
- Moving to EKS should be a new values file plus Terraform, with no app changes.

Rejected

- **Going straight to EKS:** Cost and setup before learning anything about the app on Kubernetes.

Costs and consequences

- kind could not prove IAM, the load balancer, TLS or DNS. Those were proven on 2026-10-07 when the same chart went to EKS with one values file (ADR 0014), and the cluster was retired the same day for cost (ADR 0015).

ADR 0006: One composition for preview and export; edits as a document

“Preview and export use the same React component, so the video always matches the screen, and the edit is a JSON document changed by pure functions, which makes undo, autosave and tests simple.”

Problem. A phone preview and a server render usually drift apart (text wrapping, filters, timing). The edit also has to be stored, saved and undone safely.

Decision

- One React component, `ReelComposition`, draws a reel. The editor shows it with Remotion's Player; the worker renders the same component in headless Chrome.
- Fonts ship with the bundle, filters are CSS, motion is frame-based, so the export matches the preview. Only video decoding differs (<video> on phones, WebCodecs in the worker).
- The edit is a zod-validated JSON timeline changed only by pure functions: undo/redo as snapshots, autosave guarded by a revision number, unit tests without a browser.
- Export freezes the document into the job, so later edits don't change that video and a retry renders the same thing.
- Instagram rules (3 s to 3 min, caption limits, safe zones) live in one module, checked in the editor and again on the server.

Why

- What you see is what you export.
- Every editing rule is a pure, testable function.

Rejected

- **In-browser export:** Slow and unreliable on mid-range phones, stops when the screen locks, no job to retry.
- **FFmpeg filter graphs with a separate canvas preview:** Every visual feature written twice and kept in step.
- **Event-sourced operations:** Much more machinery than a one-editor-per-reel app needs.

Costs and consequences

- Remotion needs a company licence above three people.
- The worker image is large (Chrome + compositor, ~1.5 GB).
- Signed media URLs last an hour; long sessions need a reload.
- No concurrent editing; the revision check deliberately refuses conflicting saves.

ADR 0007: Splat flythroughs rendered offline (experimental)

“The heavy 3D work runs offline on a GPU and comes back as plain video, so the editor stays simple.”

Problem. A 360 photo can only turn on the spot; a flythrough needs a 3D model of the room.

Decision

- Gaussian splatting (nerfstudio splatfacto) runs offline on a GPU as a script, outside the app.
- It renders a 1080×1920 MP4 along a smoothed camera path, which is imported as an ordinary video clip.

Why

- Best picture quality; the editor needs no 3D code because the result is just video.

Rejected

- **Depth from one 360 photo:** The camera can only move about half a metre before holes appear.
- **Box room from the floor plan:** Furniture painted flat on the walls.

Costs and consequences

- Needs a dedicated capture (100+ photos) and a GPU; experimental, not part of the deployed stack.

ADR 0008: Walking between 360 photos

“I project each 360 photo onto its room so the camera can walk between photos in one continuous move, using only the photos agents already have.”

Problem. A tour of 360 photos joined by fades reads as a slideshow; the goal is one continuous camera move.

Decision

- Project each 360 photo onto its room's shape (outline, camera and ceiling height from the floor plan).
- A 'walk' transition moves the camera from the previous photo to the next through the doorways, blending both projections; the first frame matches the previous clip, so there is no cut.
- Route and motion are pure functions in packages/core/src/walk.ts, shared by the renderer, the plan overlay and the tests.

Why

- Works with the one or two photos per room that agents already have.

Rejected

- **Gaussian splatting for every tour:** Needs far more photos per room than a normal tour has.

Costs and consequences

- Looks like Street View between photo positions; furniture is flat on the room shell.

ADR 0009: Whole-home splat from 360 photos (experimental)

“An experiment: I supply depth from a model and the floor plan so a splat can be trained from very few photos.”

Problem. A real 3D splat of a home from only one or two 360 photos per room, which have almost no overlap.

Decision

- Cut each 360 photo into ordinary views with exact poses, estimate depth with a model, fit it to the known room shapes, and seed a splat with those points.
- Train with positions frozen so the splat can't invent geometry, then film it along the same route as the 360 walk.

Why

- Real depth inside rooms from existing tour photos.

Rejected

- **Training a splat freely:** It fills the unseen space between photos with floating blobs.

Costs and consequences

- Smears at doorways and softer than the 360 walk; experimental research tooling, not production.

ADR 0010: Transactional outbox for render queue messages

“The job and its queue message are written in one database transaction and the message is sent afterwards, so a job can never be saved without its message; duplicates are fine because the worker is idempotent.”

Problem. Queuing a render writes to two systems, a job row in Postgres and a message in SQS, and they cannot share a transaction. If the web process died between the commit and the send, the job stayed QUEUED with no message.

Decision

- The message is written to an OutboxMessage table in the same transaction as the job (new job or manual retry), so they commit together or not at all.
- A relay sends pending rows: lock one with `SELECT ... FOR UPDATE SKIP LOCKED`, send it to SQS, delete it. A failed send leaves the row and retries with backoff (1 s, 2 s, 4 s, up to 60 s).
- The web app runs the relay right after the commit, so renders start as fast as before; every worker also runs it every 5 s to send what the web app could not.

Why

- A committed job always gets its message, even if the web process dies or the queue is down.
- `SKIP LOCKED` lets every web and worker process relay at once without sending the same row twice.

Rejected

- **Send first, then insert the job:** A worker could receive a message for a job that does not exist yet, or never will.
- **Change data capture (Debezium):** The standard answer at scale, but a whole extra system for a few messages a minute.
- **A separate relay deployment:** One more thing to deploy and monitor; the worker already has both connections.

Costs and consequences

- The relay can send a message twice (crash after the send, before the delete commits). The worker is already idempotent, so that is harmless.
- While the queue is down a job waits in QUEUED instead of failing straight away.
- A row lock is held during the SQS call, so the call is limited to 10 s.

ADR 0011: Terraform owns the AWS account

"I put the whole AWS account under Terraform, importing what already existed, and gave CI an OIDC role instead of keys."

Problem. When the AWS account came back from a suspension it had a hand-made media bucket and a disabled CloudFront distribution, and nothing else the app needed: no queue, no image registry, no CI identity, no runtime identity, no spending limit.

Decision

- Every AWS resource ReelWalk uses is in infra/terraform, with state in a versioned S3 bucket and S3 lock files (no DynamoDB table).
- The two resources that already existed were imported, not recreated, so the CloudFront domain stayed the same.
- A GitHub OIDC provider and a role that only the main branch of the repository can assume, limited to pushing two ECR repositories. CI holds no AWS keys.
- An application IAM policy (objects in the bucket, the two queues) that every runtime reuses: an IAM user locally, Pod Identity on EKS, task and instance roles on Fargate and App Runner.
- A monthly budget with email alerts; scanner findings about a WAF and customer-managed keys accepted as cost decisions, with reasons in .trivyignore.yaml.

Why

- terraform plan is the drift check; nothing is changed in the console.
- Importing kept the domain and avoided re-uploading media.

Rejected

- **Click it together in the console:** No history, no review, no way to rebuild it.
- **Stored access keys in GitHub secrets:** OIDC gives short-lived credentials scoped to one branch and two repositories.

Costs and consequences

- Imported resources need their exact settings written down before plan is clean.
- The IAM user is a stop-gap for local runs against real AWS.

ADR 0012: Accounts with passwords and database sessions

“Email and password with scrypt, sessions as database rows behind an httpOnly cookie, and one function that scopes every query to the signed-in workspace.”

Problem. Every request acted as one seeded demo user. To hand the app to anyone else it needed sign-up, sign-in and separation between studios, without an email provider or OAuth app existing yet.

Decision

- Email and password, hashed with scrypt from Node's crypto (no native dependency), parameters stored with the hash.
- Sessions in Postgres: the browser holds a random token in an httpOnly cookie; the table stores only its SHA-256.
- One gate: `getCurrentUser()` resolves the session or redirects to /login, and a proxy does a cheap cookie check first. Every query is scoped to the user's workspace.
- The open starter library is flagged shared and visible to every workspace; uploads stay private.

Why

- Works over plain HTTP on a phone on the home network.
- Adding magic links or Google sign-in later is another way to call `createSession()`, not a new model.

Rejected

- **A hosted auth provider:** Another account and callback to set up before anyone could log in; fine later, not for the first version.
- **JWTs in the cookie:** Cannot be revoked; a database row can.

Costs and consequences

- No password reset until there is an email provider.
- One user per workspace: no teams yet.

ADR 0013: Venue reels, stock footage and share links

"I generalised the tour builder into venue reels with dish labels and stock footage, and added a public share link because the share sheet needs HTTPS."

Problem. The editor and the tour auto-build were built for listings. A restaurant's menu is already a shot list, but most venues have no footage, and sharing a file from the share sheet only works over HTTPS.

Decision

- Venue vibes: a complete treatment (song, look, cut length, hook, label style, call to action, caption). `buildVenueReel()` turns tapped clips, dishes and prices into a timeline cut on the beat, one dish label per clip, the details card closing with price, address and handle.
- Free stock footage from Pixabay, searched server-side; imports keep the source, licence and creator credit, and the credit joins the caption.
- A public share page per reel, `/r/<token>`, with the latest export, the caption and a Save button. No sign-in; turning the link off deletes the token.

Why

- The render composition is reused unchanged: the details card shows 'from \$12' where a listing shows the price.
- A link works from any phone and messenger, HTTPS or not.

Rejected

- **A separate restaurant product:** Same timeline, same renderer, same queue; only the builder differs.

Costs and consequences

- A share link exposes the export to anyone holding it; links are long, random and revocable.
- Stock clips are stored per workspace.

ADR 0014: Production on EKS and RDS (superseded the same day)

"I ran production on EKS and RDS for a day, which proved the chart end to end, and took it down because the bill was for idle hardware."

Problem. Share links, the Instagram OAuth callback and the videos Instagram fetches all need a public HTTPS address; the chart had only run on kind.

Decision

- A VPC in two zones with nodes in public subnets and no NAT gateway (it costs more than the database), the database in private subnets.
- EKS 1.34 with one t3.xlarge node group; pods get AWS access through EKS Pod Identity, so the cluster Secret holds no keys.
- RDS Postgres 16 on the smallest Graviton class, reachable only from the node security group.
- ingress-nginx behind a network load balancer, cert-manager with Let's Encrypt, Cloudflare DNS at the apex.

Why

- It proved the chart on a real cluster: IAM, load balancer, TLS and DNS, with a real render verified end to end at reelwalking.com.

Rejected

- **A single VM with Compose:** Would have been cheaper; the user chose EKS to exercise Kubernetes on AWS.

Costs and consequences

- About \$250 a month: \$73 control plane, \$120 node, \$17 load balancer, \$15 database, almost all of it idle. Retired after one day (ADR 0015).
- Three small surprises worth remembering: RDS 16 forces TLS (sslmode), Terraform output on Windows carries a carriage return, and a changed external Secret does not roll pods by itself.

ADR 0015: Production for under \$30 a month

“I moved the web app to App Runner and made the render worker a Fargate task the web app starts on demand, cutting the bill from about \$250 to about \$15 a month with the same images and queue.”

Problem. EKS was the wrong shape for an app with one studio and a render every now and then: a quarter of the bill was the control plane and half was an idle node.

Decision

- Web app on AWS App Runner (0.5 vCPU, 1 GB) from the same ECR image; HTTPS, custom domain and certificate come with the service, so no load balancer, ingress or cert-manager.
- Render workers as Fargate tasks started on demand: when an export is queued the web app calls `ecs:RunTask` (capped by `ECS_MAX_WORKERS`), the task drains the queue and exits after four idle minutes. Nothing bills between renders.
- Postgres on a free tier outside AWS; settings and credentials in SSM Parameter Store; task and instance roles instead of keys.
- The VPC stays for the tasks; EKS, the node group, RDS and the pod-identity role are removed.

Why

- Expected run rate \$10-15 a month, idle cost about \$5.
- The queue, outbox, heartbeat lease and dead-letter handling are unchanged; only who starts the worker changed.

Rejected

- **Keep EKS on a smaller or spot node:** The \$73 control plane and \$17 load balancer are the floor; still over \$100.
- **Remotion Lambda:** Near-zero idle cost and the real long-term answer, but it replaces the worker and needs the WebGL transitions tested under Lambda; a separate project.

Costs and consequences

- The first export after a quiet spell waits one to two minutes for a cold task (image pull, Chrome start).
- If a worker exits at the same moment a message arrives, that message waits for the next export; a scheduled check could close the gap.

7. Every component

App

TypeScript + React App

Built and running

I build the UI with TypeScript and React, and validate incoming data with zod.

How it works

- I write both the front end and the server in TypeScript.
- The screens are built from React components.
- But types are only checked at compile time.
- So data coming from outside is validated with zod.
- The timeline is stored as JSON, and a schema checks its shape.
- Types can be derived from the schema, so the definition lives in one place.
- Editing logic is written as pure functions, collected in packages/core.
- That means it can be tested with Vitest without a browser.

Why ReelWalk uses it

- Because the web app and the worker can share the same types and rules.
- JSON columns in the DB are loosely typed, so I check them with zod when reading.
- The trade-off is the effort of maintaining the schemas.
- But I judged that better than the screen crashing on broken data.

What's real

Built and used everywhere: TypeScript across apps and packages, React 19 in the web app, zod 4 schemas in packages/core and in every Server Action.

In the repo

- `packages/core/src/timeline.ts` The zod timeline schema and the pure editing functions (split, trim, move).
- `apps/web/components/editor/editor.tsx` The client editor: useReducer with an undo/redo history.
- `apps/web/components/editor/history.ts` Undo/redo as a stack of timeline snapshots.
- `packages/core/tests/timeline.test.ts` Vitest tests for the editing rules, no browser needed.

Questions you may get

Q: Why do you use zod?

TypeScript types disappear at runtime. But data coming from the browser can't be trusted. So I validate it with zod at the entry of each Server Action. JSON read from the DB is checked with the same schema. Types are derived from the schema, so types and validation never drift.

Tip: They want to hear that you know types are compile-time only. Mention `z.infer`: one source for both the type and the check.

Q: How do you manage the editor's state?

The timeline is held in a useReducer. Every edit is a pure function from packages/core. Undo and redo are a stack of snapshots. A slider drag is merged into a single undo step. The playback time is not kept in React state. That avoids re-rendering the editor at 30 fps.

Tip: The playhead-outside-state detail shows real React performance experience. Say it if you have time.

Q: How much do you use `any`?

Basically never. Values from outside are received as unknown first. Then zod validates them and turns them into typed values. saveReel also takes the timeline as unknown and then parses it.

Tip: `unknown + parse` is the answer they hope for. `saveSchema` in `apps/web/app/actions.ts` really does use `z.unknown()`.

Words: type safety (型安全) · type inference (型推論) · runtime validation (実行時検証) · schema (スキーマ) · pure function (純粋関数) · state management (状態管理) · reducer (リデューサー) · undo / redo (元に戻す／やり直す) · component (コンポーネント) · type definition (型定義)

With the Next.js App Router, pages and server logic live in one app.

How it works

- Next.js is a framework for React.
- ReelWalk uses the App Router.
- The structure of the app folder becomes the URLs.
- Pages are basically Server Components.
- There is no REST API for the UI.
- All data changes go through Server Actions.
- Route Handlers are used only in a few places, like health checks.
- The build uses standalone output to keep the Docker image small.

Why ReelWalk uses it

- I don't have to build pages and an API separately, so there is less code.
- The whole stack is one TypeScript project.
- The trade-off is that Next.js has many conventions of its own.
- Caching behaviour also changes often between versions.

What's real

Built with Next.js 16; runs locally (dev server, Docker Compose, kind) and in production on AWS App Runner at reelwalking.com (ADR 0015).

In the repo

- `apps/web/app/page.tsx` The reels grid: an async Server Component that queries Prisma directly.
- `apps/web/app/api/health` Route Handlers kept only for Kubernetes probes (health, ready) and the timeline editor.
- `apps/web/next.config.ts` Standalone output and transpilePackages for the workspace packages.
- `apps/web/Dockerfile` The web image built from the standalone output.

Questions you may get

Q: Why did you choose the Next.js App Router?

The main reason is Server Components and Server Actions. Pages can call Prisma directly. Mutations are Server Actions too, so no API is needed for the UI. Building it alone, having less code mattered a lot.

Tip: Tie the choice to a concrete benefit (no separate API layer). Craftsman uses Server Components/Actions too, so you can say you used them in a real app.

Q: Where do you use Route Handlers?

Only when the caller is not this app's UI. `/api/health` and `/api/ready` are for Kubernetes probes. Besides that, one endpoint for the separate timeline editor. Everything else is a Server Action.

Tip: Shows you know when each tool fits: Server Actions for your own UI, Route Handlers for external callers.

Q: Did you do anything for performance?

I turned off prefetch on the links in the reels list. Editor pages are heavy, and every card was prefetching one. That filled the browser's concurrent connections, and the next tap had to wait. Signed URLs are also reused for the first half of their lifetime. Because if the URL changes, the browser reloads the video.

Tip: A real bug you found and fixed beats generic advice. Both are in the code (`prefetch={false}` in `app/page.tsx`, the `signedUrls` map in `lib/storage.ts`).

Words: framework (フレームワーク) · routing (ルーティング) · layout (`layout.tsx`, shared around pages) (レイアウト) · dynamic rendering (動的レンダリング) · revalidation (再検証) · Route Handler (ルートハンドラー) · health check (ヘルスチェック) · prefetch (プリフェッチ) · output (build output) (出力)

Render on the server with the data included; only the interactive parts go to the client.

How it works

- Server Components are components that run only on the server.
- They can be async functions and call Prisma directly.
- They send no JavaScript to the browser, so they are light.
- Client Components, on the other hand, run in the browser.
- If you need state or events, you add "use client".
- The editor page is a good example.
- The server loads the reel and its media and passes them as props.
- The editing itself is handled by the Editor Client Component.

Why ReelWalk uses it

- There is no need for an API layer just to fetch data.
- DB connection details never reach the browser.
- The trade-off is that you have to keep the boundary in mind.
- Only serializable values can be passed as props.

What's real

Built: the reels grid, editor page, export screen and exports list are async Server Components reading Postgres through Prisma; running locally and in production.

In the repo

- `apps/web/app/reels/[id]/page.tsx` Server Component loads the reel, library and floor plans, then renders the client Editor.
- `apps/web/components/editor/editor.tsx` The "use client" boundary: the interactive editor.
- `apps/web/components/auto-refresh.tsx` Tiny client component that calls `router.refresh()` every 2 s while a render is active.
- `apps/web/lib/workspace.ts` `getCurrentUser` wrapped in `React.cache()`, so one request queries the user once.

Questions you may get

Q: What's the difference between Server and Client Components?

Server Components run only on the server. They can access the DB directly, and no JS is sent to the browser. Client Components also run in the browser. I use them when I need state or interactions like clicks. In ReelWalk, data stays on the server and interaction on the client.

Tip: Give the rule and then one concrete example from your app (editor page loads data, Editor is the client component).

Q: How do you show render progress on screen?

The worker writes progress straight to PostgreSQL. The page is a Server Component, so it just shows the DB value. Only while a job is running, it calls `router.refresh` every 2 seconds. No data-fetching layer is needed on the client.

Tip: Polling via `router.refresh` is simple and honest. If asked about WebSockets or SSE, say it would be the next step at higher scale.

Q: How much did you make into Client Components?

As little as possible. Only interactive parts: the editor, the share button, the tab bar and so on. The lists and the export screen are Server Components.

Tip: "Push the client boundary down" is the standard best practice; say it in your own words.

Words: Server Component (サーバーコンポーネント) · Client Component (クライアントコンポーネント) · boundary (the "use client" line) (境界) · serialize (シリアライズ) · hydration (ハイドレーション) · bundle size (バンドルサイズ) · data fetching (データ取得) · rendering, drawing (描画) · re-render (再レンダリング)

Every update is a Server Action that checks input and permissions at the entry point.

How it works

- Server Actions let the browser call a function on the server.
- In ReelWalk, every data change is a Server Action.
- Creating a reel, saving, uploading, exporting, and so on.
- At the entry point, input is always validated with zod.
- Every query includes workspaceId as a condition.
- Saving uses a revision number.
- A save from an old revision is rejected.
- So that another tab's changes are never silently overwritten.

Why ReelWalk uses it

- I don't need a separate API for forms or fetch calls.
- Argument and return types flow straight through to the client.
- But there is a limit on the request body size.
- So large videos skip Server Actions and go straight to S3.

What's real

Built: createReel, saveReel (revision check), saveCaption, createUpload, confirmUpload, exportReel and retryRenderJob are real; there is no login yet, so the workspace comes from a seeded demo user.

In the repo

- `apps/web/app/actions.ts` Every Server Action; see saveReel for the revision check and retryRenderJob for a conditional retry.
- `apps/web/components/editor/use-autosave.ts` Calls saveReel 700 ms after an edit with the last revision it saw.
- `apps/web/lib/workspace.ts` getCurrentUser: the one place to swap in real auth later.
- `packages/core/src/instagram.ts` Instagram rules, re-checked on the server inside exportReel.

Questions you may get

Q: How did you implement the revision check?

The browser sends the last revision it saw along with the save. The server puts that revision in the where clause of updateMany. On success, the revision is incremented by one. If zero rows changed, another tab saved first. Then it doesn't overwrite, and asks the user to reload.

Tip: Name it: 楽観的ロック (optimistic locking). Mention it's one SQL statement, no read-then-write, no explicit lock.

Q: What do you watch out for with Server Actions security?

A Server Action is effectively a public endpoint. So input is always validated with zod. Every query is scoped to the workspace. On save I also check the media used belongs to the same workspace. But there's no login yet; it runs as a demo user.

Tip: Being upfront that auth is missing (and that it's isolated in one function) reads as maturity, not weakness.

Q: What does the server do on export?

First it re-checks the Instagram rules on the server. Next it saves the timeline as of that moment into the job. Later edits don't change the video being exported. In the same transaction, it also writes the queue message to the database. After the commit, it sends that message to the queue.

Tip: Freezing the payload makes retries deterministic. Link to the queue and idempotency topics if they dig in.

Words: Server Action (サーバーアクション) · mutation (更新処理) · input validation (入力検証) · optimistic locking (楽観的ロック) · conflict (競合) · overwrite (上書き) · authorization (認可) · conditional update (条件付き更新) · autosave (自動保存)

One React component draws both the live preview and the exported video.

How it works

- Remotion is a library for making video with React.
- From a frame number, it draws the picture for that moment.
- In ReelWalk, ReelComposition draws a reel.
- In the editor, it's previewed with @remotion/player.
- On export, the worker renders the same component.
- Inside, headless Chrome and ffmpeg do the work.
- So the video matches what you saw on the phone.
- The only difference is how video is decoded.

Why ReelWalk uses it

- If preview and export are built separately, they drift apart.
- With one component, each feature is written once.
- The trade-off is a large worker image, about 1.5 GB.
- Also, companies of four or more people need a paid licence.

What's real

Built: the Player preview and worker renders to MP4 both work locally and in the Playwright e2e run; ADR 0006 records the decision.

In the repo

- `packages/render/src/reel/ReelComposition.tsx` The one composition used by both the Player and the worker.
- `apps/web/components/editor/preview.tsx` The editor preview using `<Player>` from `@remotion/player`.
- `packages/render/src/render-reel.ts` Worker side: bundle, selectComposition and renderMedia to MP4.
- `docs/adr/0006-one-composition-for-preview-and-export.md` Why one composition, and the rejected alternatives.

Questions you may get

Q: Why not export on the phone itself?

It costs nothing on the server, but it's slow on mid-range phones. It stops when the screen locks. And when it fails, there's no job left to retry. So exports run on a worker on the server.

Tip: Show you weighed the cheaper option. The older desktop editor (apps/editor) does in-browser export, so you know both.

Q: Are there differences between preview and export?

Only video decoding differs. The preview uses a plain video element with the phone's hardware decoder. The worker uses `@remotion/media`, which uses WebCodecs. When I measured it, it was about twice as fast as `OffthreadVideo`. Fonts and filters are the same, so the look matches.

Tip: "I measured it" is strong. Keep the number modest and say it was a local measurement.

Q: How would you make rendering faster?

Most of the time goes into decoding video. Within one render, `RENDER_CONCURRENCY` raises the parallelism. Overall, I scale out by adding worker pods. Scaling on queue depth with KEDA is the next step.

Tip: Separate scale-up (concurrency per render) from scale-out (more pods). KEDA is planned, not built.

Words: composition (Remotion's unit of video) (コンポジション) · preview (プレビュー) · export (rendering the final file) (書き出し) · headless browser (ヘッドレスブラウザ) · frame (フレーム) · decode (デコード) · vertical video (9:16) (縦型動画) · degree of concurrency (並列度) · safe zone (安全領域)

Data

The DB definition and a typed client are shared by the web app and the worker.

How it works

- Prisma is an ORM for TypeScript.
- You write the models in `schema.prisma`.
- A typed client is generated from it.
- The schema and migrations live in `packages/db`.
- The web app and the worker use the same client.
- Migrations are committed as SQL files.
- On the cluster, a Kubernetes Job applies them.
- `prisma migrate deploy` runs before the app is updated.

Why ReelWalk uses it

- The main reason is sharing types between web and worker.
- The first version used raw SQL with `asyncpg`, but types couldn't be shared.
- The trade-off is that you get type errors unless generate runs first.
- Prisma 7 also requires a driver adapter.

What's real

Built: Prisma 7 with `@prisma/adapter-pg`, seven committed migrations, a migrate Job in the Helm chart (kind, and EKS for a day), and a one-off Fargate task that applies them in production.

In the repo

- `packages/db/prisma/schema.prisma` All models: Workspace, User, Reel, MediaAsset, Tour, RenderJob, RenderOutput.
- `packages/db/prisma/migrations` Committed SQL migrations applied with `migrate deploy`.
- `packages/db/src/index.ts` The shared client with the `pg` adapter, kept on `globalThis` in dev.
- `infra/helm/reelwalk/templates/migrate-job.yaml` The one-shot migrate Job (Argo CD `sync-wave -1`, name hashed per release).

Questions you may get

Q: How do you run migrations?

During development, `prisma migrate dev` creates the SQL and I commit it. On the cluster, a Helm Job runs `migrate deploy`. With Argo CD, this Job is made to finish first. The Job name includes a hash, so each release creates a new one. I verified this on kind.

Tip: Mention the Job pod template is immutable, which is why the name carries a hash. Say kind, not EKS.

Q: Any problems using Prisma?

The client is generated into the repository tree. If you forget generate, typecheck and tests both fail. So I made Turborepo run it first. Another was connections piling up during Next.js development. I keep the client on `globalThis` so there's only one.

Tip: Concrete pain + concrete fix. Both fixes are visible in `turbo.json` and `packages/db/src/index.ts`.

Words: object-relational mapper (ORM) · migration (マイグレーション) · typed client (型付きクライアント) · code generation (コード生成) · driver adapter (ドライバーアダプター) · seed data (シード) · connection pool (接続プール) · upsert (insert or update) (アップサート) · to apply (a migration) (適用する)

PostgreSQL is the source of truth, and state changes are conditional UPDATES.

How it works

- PostgreSQL holds the source of truth.
- Reels, media, jobs and outputs are all in it.
- Queue messages carry only the job ID and its generation.
- State is always decided by looking at the DB row.
- A status change is a single conditional UPDATE.
- The WHERE clause includes the current status and generation.
- If someone updated it first, zero rows change, that's all.
- There's no read-then-write, so no locks are needed.

Why ReelWalk uses it

- Because the data is relational and needs joins.
- I considered DynamoDB, but the access patterns weren't settled yet.
- The AWS design uses RDS; only DATABASE_URL changes.
- The trade-off is having to think about connection counts and scaling yourself.

What's real

Built: PostgreSQL 16 runs locally in Docker Compose and kind; production used RDS for a day (ADR 0014) and now uses the free Postgres tier at Supabase (ADR 0015). The conditional updates are real and tested.

In the repo

- `apps/worker/src/store.ts` Every job status change as one conditional updateMany; `succeed()` wraps `upsert + update` in a transaction.
- `packages/core/src/job-status.ts` The legal state transitions as a pure function, covered by Vitest.
- `docs/adr/0002-postgresql-and-prisma.md` Why PostgreSQL + Prisma, and why not DynamoDB or raw SQL.
- `docs/adr/0014-eks-and-rds.md` RDS Postgres 16 (db.t4g.micro, private subnets, TLS required) ran for one day; the Terraform is in git history.

Questions you may get

Q: What happens if two workers pick up the same job?

Both try to set it to RUNNING with `updateMany`. The WHERE accepts QUEUED, or RUNNING with a stale heartbeat. Only the one that writes first updates one row. The other gets zero rows, so it does nothing. The DB decides, so the app needs no lock.

Tip: This is the core of the design. Draw it on paper if there's a whiteboard: WHERE status/generation is the guard.

Q: Do you use transactions?

On completion, saving the output and updating the status are one transaction. But the database and the queue are separate systems, so they can't share one transaction. So I use the outbox pattern. In the same transaction as the job, I write the message to a table in the database. After the commit, it is sent from that table to the queue. If the send fails, the message stays in the database, so it can be sent again later.

Tip: Know the why: the DB and the queue can't share a transaction, so the message goes into the DB first (ADR 0010). The relay can send twice, which the idempotent worker already handles.

Q: What do you do for the database in production?

At first I created RDS PostgreSQL in private subnets with Terraform. It ran for a day, and I retired it along with EKS over cost. Now I use a free PostgreSQL tier outside AWS. Locally it runs on the same PostgreSQL 16.

Tip: RDS was real for one day (ADR 0014). Say why it went: \$15 a month for an idle database when a free tier does the job (ADR 0015). Mention the one surprise: RDS 16 forces TLS, so the connection string needs `sslmode`.

Words: relational database (リレーショナルデータベース) · conditional update (条件付き更新) · race condition (競合状態) · mutual exclusion / concurrency control (排他制御) · optimistic locking (楽観的ロック) · transaction (トランザクション) · generation (the retry counter on a job) (世代) · source of truth (正のデータ) · managed service (like RDS) (マネージドサービス)

Where uploads and rendered videos live; the browser uploads straight to it with presigned URLs.

How it works

- S3 is AWS's object storage.
- It holds uploaded media and the exported MP4s.
- The browser uploads directly to S3.
- The server only issues a presigned URL.
- The upload URL expires after 10 minutes.
- Then HeadObject checks that the file is really there.
- Playback and downloads use presigned GET URLs.
- Locally, MinIO stands in for S3.

Why ReelWalk uses it

- Because I don't want 2 GB videos passing through Next.js memory.
- The code stays on the AWS SDK; only the endpoint changes.
- The trade-off is that URL signing and endpoint settings get a bit fiddly.
- Because the in-cluster address differs from the one the browser can reach.

What's real

The code uses the AWS SDK v3 S3 client and presigned URLs, but it has only run against MinIO (Docker Compose and kind); the S3 bucket and CloudFront exist only in the Terraform sketch.

In the repo

- `apps/web/lib/storage.ts` Presigned PUT/GET, HeadObject, separate in-cluster and public endpoints, and the signed-URL reuse cache.
- `packages/core/src/keys.ts` Object key layout: uploads/, thumbs/, and renders/<jobid>.mp4 for idempotent output.
- `apps/web/components/editor/upload.ts` The browser side: PUTs the file and thumbnail straight to storage.
- `docker-compose.yml` MinIO (Chainguard image) and ElasticMQ as local stand-ins.

Questions you may get

Q: Why upload directly from the browser?

Videos can be up to 2 GB. There's no reason to pass them through the web server's memory. Server Actions also have a body size limit. So the server only validates and returns a presigned URL. After the upload, it confirms the object with HeadObject before recording it.

Tip: Mention the two-step flow (createUpload, then confirmUpload). The server never decodes media; the browser measures duration and makes the thumbnail.

Q: Any gotchas with presigned URLs?

Yes, a few. First, the signature must be made for the host the browser can reach. So I made two clients: one for server calls, one for signing. Next, the browser can't send the SDK's checksum headers. Finally, each signing gives a new URL, which made videos reload.

Tip: Specific war stories show you really built it. All three fixes are in `apps/web/lib/storage.ts`.

Q: Did you actually use AWS S3?

Yes, production runs on real S3 and CloudFront. I imported the bucket into Terraform and added CORS and encryption. Locally I use MinIO, with the same AWS SDK. Switching is just a change of environment variables.

Tip: Honesty is the point here. Then pivot to what you did verify locally.

Words: object storage (オブジェクトストレージ) · bucket (バケット) · object key (オブジェクトキー) · presigned URL (署名付き URL) · expiry / validity period (有効期限) · direct upload (直接アップロード) · endpoint (エンドポイント) · S3-compatible (S3互換) · idempotency (冪等性) · to issue (a URL, a token) (発行する)

Queue and render

The queue that hands render jobs from the web app to the workers.

How it works

- Rendering a video can take several minutes.
- So we don't do it inside a web request.
- The web app writes the job to the database, then sends a message to SQS.
- The message carries only the jobId and the generation.
- Workers receive with long polling, waiting up to 10 seconds.
- A received message stays invisible for the visibility timeout.
- That effectively works as a lease on the job.
- Locally, ElasticMQ provides the same API.

Why ReelWalk uses it

- The first version used a Redis list, and a job was lost whenever a worker died.
- SQS has a visibility timeout and a dead-letter queue out of the box.
- The trade-off is at-least-once delivery, so duplicates have to be handled.
- Using Postgres as a queue was an option, but it fits poorly with long renders.

What's real

The code uses the real AWS SDK SQS client against ElasticMQ locally (Docker Compose and kind) and against SQS in production, where Terraform creates the queue and its dead-letter queue with the same lease and receive count (ADR 0011, 0015).

In the repo

- `apps/web/lib/queue.ts` `SendMessage` with a body of only `{ jobId, generation }`.
- `packages/db/src/outbox.ts` The outbox: the message is written with the job in one transaction, then relayed to SQS (`FOR UPDATE SKIP LOCKED`).
- `apps/worker/src/index.ts` `ReceiveMessage` with `WaitTimeSeconds 10`, `MaxNumberOfMessages 1` and the visibility timeout.
- `docker-compose.yml` ElasticMQ config: render-jobs, 120 s visibility, DLQ after 3 receives.
- `docs/adr/0001-sqs-for-render-jobs.md` Why SQS over a Redis list, Postgres `SKIP LOCKED` or one Kubernetes Job per render.

Questions you may get

Q: Why did you choose SQS?

Renders are long and the process can die halfway. With the first Redis list, the job was lost when that happened. With SQS the visibility timeout means the job is redelivered after a crash. I don't have to build receive counts or a DLQ myself.

Tip: They want a reason tied to failure handling, not 'it is the AWS default'. Mention the Redis LPOP version you replaced.

Q: How long is the visibility timeout?

120 seconds. While rendering, the worker extends it every 20 seconds. So the message stays hidden even during a long render. If the worker dies, extending stops and the message is back within two minutes.

Tip: Show you extend a short timeout (`ChangeMessageVisibility`) rather than setting one huge timeout; a long fixed timeout means slow recovery after a crash.

Q: Don't you use a FIFO queue?

No, it is a standard queue. Order doesn't matter for renders. Even with FIFO, the same message comes back if a worker dies. So the worker has to be idempotent in any case.

Tip: FIFO deduplication only covers sends within 5 minutes; it does not stop redelivery after a crash.

Words: queue (キュー) ・ message queue (メッセージキュー) ・ long polling (ロングポーリング) ・ visibility timeout (可視性タイムアウト) ・ receive count (受信回数) ・ at-least-once delivery (少なくとも一回の配信) ・ standard queue (標準キュー) ・ loose coupling (疎結合) ・ lease (リース)

It takes jobs from the queue, renders the video with Remotion and puts it in S3.

How it works

- The worker is a Node.js process separate from the web app.
- It long-polls SQS and receives one message at a time.
- First it looks at the job row and decides whether to process it.
- It claims the job by setting it to RUNNING with a conditional UPDATE.
- Remotion renders it with headless Chrome and ffmpeg.
- Every 20 seconds it extends the timeout and writes a heartbeat.
- It uploads the MP4 to S3, sets SUCCEEDED and deletes the message.
- Each pod renders only one job at a time.

Why ReelWalk uses it

- A render uses gigabytes of memory; its load looks nothing like the web app's.
- Kept apart, a crashing render never takes the site down.
- One render per pod makes memory easy to size.
- The trade-off is that throughput only grows by adding pods.

What's real

Built and tested (Vitest handler tests, a real render in Playwright); it runs in Docker Compose and as 2 pods on kind against ElasticMQ and MinIO, never on EKS. In production the real AWS service is used (ADR 0011, 0015).

In the repo

- `apps/worker/src/index.ts` Poll loop, /healthz liveness endpoint and SIGTERM handling.
- `apps/worker/src/handler.ts` handleDelivery: decide, claim, heartbeat, render, succeed or retry.
- `apps/worker/src/config.ts` Checks heartbeat 20 s < stale 60 s < visibility 120 s at startup.
- `infra/helm/reelwalk/templates/worker.yaml` Deployment with a 300 s grace period, /dev/shm for Chrome, optional CPU HPA.

Questions you may get

Q: What happens if a worker dies mid-render?

Extending the timeout and the heartbeat both stop. After 120 seconds the message becomes visible again. Another worker receives it and takes over, because the heartbeat is stale. The output overwrites the same key, so there is still one video.

Tip: Quote the timing chain: heartbeat 20 s < stale 60 s < visibility 120 s. `config.ts` refuses to start otherwise, so a redelivery is never wasted on a 'defer'.

Q: What happens to a render during a deploy?

On SIGTERM it stops taking new messages. It carries on with the current render. The grace period is 300 seconds. If it runs out, another pod redoes the job after the timeout.

Tip: This is graceful shutdown plus at-least-once as a safety net. Saying both shows you thought about rollouts, not just happy paths.

Q: Why only one render per pod?

Rendering in Chrome uses a lot of memory. Limiting it to one makes resource limits easy to set. Inside one render, it uses four Chrome tabs in parallel. Today's HPA is CPU-based; scaling on queue depth (KEDA) is the planned improvement.

Tip: HPA is off by default in `values.yaml` and KEDA is only planned. Say 改善点 / 予定, not 使っています.

Words: worker (ワーカー) · export / render (書き出し) · heartbeat (ハートビート) · extension (延長) · acquire / claim (取得) · graceful shutdown (グレースフルシャットダウン) · grace period (猶予期間) · liveness probe (ライブネスプローブ) · scale out (スケールアウト)

One table defines a job's states and which changes between them are allowed.

How it works

- There are four states: QUEUED, RUNNING, SUCCEEDED and FAILED.
- A new job is QUEUED; once a worker takes it, RUNNING.
- SUCCEEDED on success; FAILED when attempts run out.
- If an attempt fails with attempts left, it goes back to QUEUED.
- From FAILED, the user can retry by hand in the UI.
- The allowed transitions are written in a table in packages/core.
- Each update is conditional, with the expected current state in the WHERE clause.
- It isn't a read followed by a write, so there is no race.

Why ReelWalk uses it

- As a table, every transition can be checked in tests.
- Conditional updates give mutual exclusion without holding a lock for long.
- The catch is that the table and each WHERE clause must be kept in step by hand.

What's real

Fully built: the transition table and its tests live in packages/core, and every status write in the worker and the Server Actions is a conditional Prisma updateMany on Postgres.

In the repo

- `packages/core/src/job-status.ts` The TRANSITIONS table, canTransition and isTerminal.
- `apps/worker/src/store.ts` claim, heartbeat, succeed, requeue and fail as conditional updateMany calls.
- `apps/web/app/actions.ts` retryRenderJob: FAILED -> QUEUED and generation + 1 in one write.
- `packages/core/tests/job-status.test.ts` Vitest tests for legal and illegal transitions.

Questions you may get

Q: What if two workers grab the same job at the same time?

Claiming is a single UPDATE statement. The condition is: QUEUED and the same generation. The database applies them in turn, so only one succeeds. The other updates zero rows, leaves the message and stops.

Tip: The key point is that the check and the write are one atomic statement, not SELECT then UPDATE.

Q: Why go back from RUNNING to QUEUED?

Because the job is waiting for an automatic retry. The UI can show that it is waiting. The next worker can claim a QUEUED job the normal way.

Tip: Mention that requeue also clears heartbeatAt, so a retried job never looks like it is still being rendered.

Q: Can the state change after SUCCEEDED?

No. It is a terminal state. A message that arrives late is simply deleted. Exporting again creates a new job.

Words: state transition (状態遷移) · state machine (ステートマシン) · conditional update (条件付き更新) · conflict / race (競合) · mutual exclusion / concurrency control (排他制御) · optimistic locking (楽観的ロック) · terminal state (終端状態) · generation (世代)

However many times a message is processed, there is one result.

How it works

- SQS delivers at least once.
- So the same message can arrive twice.
- When a worker dies, the same job arrives again too.
- So I made processing give one result no matter how many times it runs.
- Messages for finished jobs or an old generation are just deleted.
- If another worker is rendering it, the message is left alone.
- The output key is derived from the job ID, so a rerun overwrites it.
- The output row has a unique jobId and is written with an upsert.

Why ReelWalk uses it

- Exactly-once delivery is hard in a distributed system.
- Making the receiver idempotent is the simplest answer.
- The trade-off: occasionally rendering the same video twice, which I accept.

What's real

Built and covered by Vitest handler tests (duplicate delivery, stale generation, take-over of a dead worker's job); exercised locally with ElasticMQ, never under production load. Per-claim fencing tokens are not implemented. The transactional outbox is built (ADR 0010) and tested against a real Postgres.

In the repo

- `packages/core/src/queue.ts` `decideDelivery`: claim, discard or defer, from the job row and the message generation.
- `apps/worker/src/store.ts` `RenderOutput` upsert and status writes conditional on generation.
- `apps/worker/tests/handler.test.ts` Tests for double delivery, stale generations and stale heartbeats.
- `docs/adr/0003-separate-idempotent-render-workers.md` The five rules that make a second render harmless.

Questions you may get

Q: How do you prevent duplicate messages?

I don't prevent them; I make them harmless. On arrival, the worker first checks the job's state and generation. If it is already done, it just deletes the message. Even if it renders twice, the output has the same key and there is one row.

Tip: Reframing 'prevent' as 'make harmless' is exactly what the interviewer hopes to hear for an at-least-once queue.

Q: What if an old message arrives after a manual retry?

Each retry bumps the generation by one. The message carries a generation too. If it doesn't match, it is deleted as stale. The retry itself is a conditional update, so tapping twice counts once.

Q: Are there still weak spots?

When a worker judged dead is actually still alive. Then two workers can render the same job. The output is the same, so the result isn't corrupted. To improve it, I'd issue a token per claim and fence writes with it. The gap between the DB write and the send was a weak spot too, but I closed it with an outbox.

Tip: Naming a real gap plus a concrete fix beats claiming perfection. Be clear the fencing token is an improvement, not built; the outbox is built (ADR 0010).

Words: idempotency (冪等性) · duplicate (重複) · deduplication (重複排除) · at least once (少なくとも一回) · overwrite (上書き) · unique constraint (ユニーク制約) · generation (世代) · side effect (副作用) · fencing token (フェンシングトークン)

A failure is retried with growing gaps, and after 3 tries goes to the DLQ.

How it works

- When a render fails, it doesn't give up right away.
- The job goes back to QUEUED and the message is kept.
- The visibility timeout is used as the wait.
- The wait doubles, 15 s then 30 s: exponential backoff.
- If the third attempt fails, the job is set to FAILED.
- The next receive exceeds the limit, so SQS moves it to the DLQ.
- A loop reading the DLQ fails any job still left active.
- A FAILED job can be retried by hand from the UI.

Why ReelWalk uses it

- Temporary failures often fix themselves after a short wait.
- With a DLQ, a broken message doesn't loop forever.
- But the max count has to match in two places: the worker and the queue.
- There's no jitter yet; that is an improvement to make.

What's real

Backoff retries, the 3-receive redrive and the DLQ consumer are built and tested, and run against ElasticMQ's dead-letter queue locally; the real SQS redrive policy is not in Terraform yet. In production the real AWS service is used (ADR 0011, 0015).

In the repo

- `packages/core/src/queue.ts` `retryDelaySeconds`, `decideFailure` and `shouldFailFromDeadLetter`.
- `apps/worker/src/handler.ts` `Requeue` with `extend(delay)`, `extend(0)` on the last attempt, `handleDeadLetter`.
- `docker-compose.yml` `ElasticMQ` `deadLettersQueue` with `maxReceiveCount = 3`.
- `apps/web/app/actions.ts` `retryRenderJob`, the manual retry from the UI.

Questions you may get

Q: How many retries, and at what intervals?

Up to three attempts in total. The gap doubles: 15 seconds, then 30. It is capped at 15 minutes. The wait is made with the message's visibility timeout.

Tip: A neat detail: no scheduler is needed; `ChangeMessageVisibility` hides the message for exactly the backoff.

Q: What if the worker dies every time and never writes FAILED?

Say the pod is killed for running out of memory every time. After three receives, the message moves to the DLQ. The worker also reads the DLQ and fails the job if it still looks active. So a job never stays RUNNING forever.

Tip: This case is the reason the DLQ consumer exists. It is tested in `apps/worker/tests/handler.test.ts`.

Q: Couldn't retries make the load worse?

Attempts are limited to three, so it can't grow without bound. The gap doubles, so retries don't pile up at once. But with no jitter, jobs that fail together retry together. I'd improve that by adding a random spread.

Words: `retry` (リトライ) · `exponential backoff` (指数バックオフ) · `jitter` (ジッター) · `dead-letter queue` (デッドレターキュー) · `max receive count` (最大受信回数) · `redrive` (リドライブ) · `poison message` (ポイズンメッセージ) · `transient failure` (一時的な障害) · `manual retry` (手動リトライ) · `to give up` (諦める)

Platform

We package each app as a Docker image so it runs the same way everywhere.

How it works

- We build one image per deployable unit.
- web is the Next.js app; worker is the Node process that renders videos.
- Both use multi-stage builds that separate the build stage from the runtime stage.
- web uses Next.js standalone output and copies only the files the server needs.
- The worker image contains Chrome and ffmpeg, and renders MP4s.
- Chrome is downloaded at build time, so a new pod can start working right away.
- The package managers such as npm are removed from the runtime images.
- Finally, USER node makes them run as a non-root user.

Why ReelWalk uses it

- Splitting by deployable means each image holds only what it needs.
- The worker is heavy because it contains Chrome, but web stays light.
- The trade-off is two Dockerfiles to maintain.

What's real

Real: the web and worker images build in CI on every push; on main they are pushed to GHCR and to ECR (GitHub OIDC role, no stored keys). App Runner and Fargate run the ECR images in production.

In the repo

- `apps/web/Dockerfile` Three stages: deps, builder, and a runner that copies only the standalone output and runs as node.
- `apps/worker/Dockerfile` Chrome libraries and ffmpeg in the base, a --prod install, Chrome downloaded at build time, tsx started directly so SIGTERM arrives.
- `apps/web/next.config.ts` output: "standalone" with the monorepo root as the tracing root.
- `.dockerignore` Keeps node_modules, build output and local data out of the build context.

Questions you may get

Q: Did you do anything to keep the images small and safe?

With multi-stage builds, no build tooling is left in the final image. The worker installs production dependencies only. I also removed npm and corepack, which are unused, from the runtime image. That cleared Trivy's HIGH and CRITICAL findings. And it runs as the node user, not root.

Tip: Give a concrete before/after: Trivy flagged the bundled npm, the pnpm store and old esbuild builds; removing them (commit c317ca1) cleared the image scan.

Q: Why separate images for web and worker?

Because they need completely different things. The worker needs Chrome and ffmpeg, which makes the image heavy. Putting that into web would slow down its deploys and startup. Kept apart, they can also scale independently.

Tip: Link it to the architecture: rendering runs in separate workers (ADR 0003), so the image split follows the process split.

Q: Anything you were careful about when containers stop?

The worker starts tsx directly. Through a package manager, SIGTERM would not reach it. On SIGTERM it finishes the current render, then stops. The Kubernetes grace period is set to 300 seconds.

Tip: Graceful shutdown is a classic follow-up. Mention `terminationGracePeriodSeconds: 300` in the chart values.

Words: container (コンテナ) · image (イメージ) · multi-stage build (マルチステージビルド) · base image (ベースイメージ) · layer cache (レイヤーキャッシュ) · non-root user (非rootユーザー) · dependencies (依存関係) · attack surface (攻撃対象領域) · making something smaller / lighter (軽量化) · export / render (of a video) (書き出し)

It manages containers declaratively and brings them back automatically when they fail.

How it works

- Kubernetes is the platform that runs containers together.
- The smallest unit is the Pod.
- web and worker are separate Deployments.
- web has two replicas and is reached through a Service.
- The worker pulls work from the queue, so it has no Service.
- web has two probes: /api/ready and /api/health.
- Every container has resource requests and limits.
- Database migrations run as a Job on each release.

Why ReelWalk uses it

- ReelWalk has three workloads with different needs.
- web that updates without downtime, workers that scale out, and a run-once migration.
- At this size ECS, which is easier to run, would also be a reasonable choice.
- I still chose Kubernetes because the same manifests run locally and on EKS.

What's real

The Deployments, Service, probes, resources and migration Job are real: they run on a local kind cluster and ran on EKS for one day (ADR 0014) before production moved to App Runner and Fargate for cost (ADR 0015).

In the repo

- `infra/helm/reelwalk/templates/web.yaml` web Deployment with `maxUnavailable: 0`, readiness on `/api/ready`, liveness on `/api/health`, plus the Service.
- `infra/helm/reelwalk/templates/worker.yaml` worker Deployment: no Service, liveness on `/healthz`, `/dev/shm` in memory for Chrome, 300 s grace period.
- `infra/helm/reelwalk/templates/migrate-job.yaml` The migration Job, named with a hash because a Job's pod template is immutable.
- `docs/adr/0004-kubernetes-and-helm.md` Why Kubernetes, why the probe split, and the ECS alternative.

Questions you may get

Q: What is the difference between readiness and liveness?

Liveness checks whether the process is alive. If it fails, the container is restarted. Readiness checks whether it can take traffic. If it fails, the pod is only taken out of the Service. In ReelWalk only readiness checks the database. That way a database outage does not cause a restart loop.

Tip: The point they want: a dependency outage should remove pods from rotation, not restart them. `/api/ready` also returns 503 until the seed has run.

Q: Why did you choose Kubernetes?

Because there were three workloads with different needs. Rolling updates, scaling and Jobs are all built in. ECS is easier to operate, and I considered it. But the same chart runs on both kind and EKS. For now I have verified it on a local kind cluster.

Tip: Naming the alternative (ECS/Fargate) shows judgment, and the twist is that production ended up on Fargate after all: EKS ran for a day, cost about \$250 a month, and was retired (ADR 0014, 0015). Kubernetes stays the local demo.

Q: How do you deploy without downtime?

A rolling update with `maxUnavailable` set to 0. Old pods stop only after new ones are ready. Migrations run as a Job. Readiness waits for the migration to finish. Workers finish their current render before stopping.

Tip: Mention that migrations must stay backward compatible, because old and new pods run side by side during the rollout.

Q: How did you set requests and limits?

Requests are what the scheduler uses to place pods. Limits are set on memory only. CPU is left unlimited so pods can use spare capacity. The worker gets more memory because it runs Chrome. But the values are estimates, not production measurements.

Tip: Memory over the limit means OOMKilled; CPU over a limit means throttling. Be clear the numbers are not from production load.

Words: Pod (ポッド) · Deployment (the resource) (デプロイメント) · Service (サービス) · replica (レプリカ) · readiness probe / liveness probe (レディネスプローブ / ライブネスプローブ) · health monitoring (alive or dead) (死活監視) · resource requests and limits (リクエストとリミット) · rolling update (ローリングアップデート) · declarative (宣言的) · grace period (猶予期間)

It packages Kubernetes manifests as templates plus values.

How it works

- Helm is the package manager for Kubernetes.
- A package is called a chart.
- ReelWalk's chart is in `infra/helm/reelwalk`.
- The templates are `web`, `worker` and the `migration Job`.
- Per-environment differences go into values files.
- `values.yaml` holds the defaults for AWS.
- `values-kind.yaml` overrides them for the local kind cluster.
- CI runs `helm lint` against both combinations.

Why ReelWalk uses it

- The differences between environments show up clearly in one file.
- With one environment, plain manifests or Kustomize would be enough.
- The downside is that template syntax is hard to read.

What's real

Real: the chart is linted in CI and installed on the local kind cluster with `values-kind.yaml`; `values.yaml` (the AWS defaults) has never been installed anywhere.

In the repo

- `infra/helm/reelwalk/values.yaml` Defaults for a real cluster: AWS endpoints empty, `secret.create false`, IRSA annotation slot.
- `infra/helm/reelwalk/values-kind.yaml` kind overrides: NodePort 30080, MinIO and ElasticMQ endpoints, throwaway credentials.
- `infra/helm/reelwalk/templates/_helpers.tpl` Shared labels, the config checksum, and the non-root, read-only security contexts.
- `infra/helm/reelwalk/templates/migrate-job.yaml` Job name hashed from the image tag and chart version.

Questions you may get

Q: What is the difference between `values.yaml` and `values-kind.yaml`?

`values.yaml` assumes real S3 and SQS. `values-kind.yaml` points the endpoints at MinIO and ElasticMQ. The Service becomes a NodePort, reachable on `localhost:8081`. The throwaway local credentials are there too. The application code has no local-only branches.

Tip: Key message: the app always speaks the S3/SQS APIs; only configuration differs between environments.

Q: How are pods updated when configuration changes?

A checksum of the ConfigMap and Secret is put into an annotation. When the settings change, so does the checksum. That changes the pod template, which triggers a rolling update.

Tip: `envFrom` values are read only at container start, so without this trick a config change would not reach running pods.

Q: Why does the migration Job name include a hash?

Because a Job's pod template cannot be changed afterwards. With the same name, the upgrade would fail. The hash comes from the image tag and chart version. So each release creates a new Job.

Tip: Shows you have hit a real Kubernetes gotcha (immutable fields), not just read about Helm.

Words: chart (チャート) · template (テンプレート) · values file (`values` (バリュース)) · release (an installed chart) (リリース) · override (上書き) · template / boilerplate (雛形) · checksum (チェックサム) · ConfigMap and Secret (ConfigMapとSecret) · immutable (不変)

It creates a Kubernetes cluster inside Docker containers.

How it works

- kind stands for Kubernetes in Docker.
- Each node runs as a Docker container.
- up.sh goes from creating the cluster to deploying in one run.
- kind nodes cannot see host images, so the script loads them in.
- Postgres, MinIO and ElasticMQ start outside the chart.
- Because in production they become RDS, S3 and SQS.
- A port mapping exposes web on localhost:8081.

Why ReelWalk uses it

- EKS costs money from the first hour.
- VPC and IAM work is also needed before the first pod.
- kind can be created and destroyed in minutes, at no cost.
- But it cannot verify things like IRSA or load balancers.

What's real

Real: infra/kind/up.sh creates the cluster, loads the images and installs the chart locally; kind is the stand-in for EKS, which was never created.

In the repo

- `infra/kind/cluster.yaml` One control-plane node; maps NodePort 30080 to localhost:8081 and 30900 (MinIO) to 9100.
- `infra/kind/up.sh` Idempotent bootstrap: create cluster, build and kind-load images, apply deps, helm upgrade --install.
- `infra/k8s/local/deps.yaml` Postgres, MinIO and ElasticMQ as local stand-ins for RDS, S3 and SQS.
- `docs/adr/0005-local-kubernetes-before-eks.md` Why kind first, and the list of what kind does not prove.

Questions you may get

Q: Why kind instead of EKS?

The goal was to learn and demonstrate operating Kubernetes. EKS costs money and needs VPC and IAM setup. kind is free and quick to recreate. The differences from AWS are collected in values-kind.yaml. But I have not yet verified that it really runs on EKS.

Tip: Saying exactly what is not yet proven builds trust. ADR 0005 has the list.

Q: What can you not verify on kind?

Real IAM permissions, such as IRSA. There is no load balancer, TLS or DNS either. Node autoscaling is also out of scope. Same for RDS networking and backups. Those would first be verified on EKS.

Tip: A strong answer lists concrete gaps rather than saying 'it is basically the same'.

Words: cluster (クラスター) · node (ノード) · control plane (コントロールプレーン) · port mapping (ポートマッピング) · NodePort Service type (NodePort (ノードポート)) · loading an image into the cluster (イメージのロード) · idempotent (冪等) · substitute / stand-in (代替) · test / verification environment (検証環境)

CI/CD

Runs the monorepo's tasks in dependency order, with caching.

How it works

- Turborepo is a task runner for monorepos.
- apps and packages are managed as a pnpm workspace.
- It runs typecheck, test and build in dependency order.
- Results are cached, so with no changes it finishes instantly.
- The key point is running db:generate first.
- Without the Prisma client, the type check fails.
- turbo.json's dependsOn encodes that order.

Why ReelWalk uses it

- Because web and worker share the core, db and render packages.
- Managing the order by hand invites mistakes.
- The trade-off is that a wrong config can serve a stale cache.
- db:generate declares its inputs, so it regenerates only when the schema changes.

What's real

Built: `pnpm typecheck/test/build` run through turbo locally and in GitHub Actions; caching is local only (no remote cache), and the Dockerfiles do not use turbo prune.

In the repo

- `turbo.json` The task graph: typecheck and test depend on db:generate and ^db:generate.
- `pnpm-workspace.yaml` Workspace members: apps/*, packages/*, e2e.
- `package.json` Root scripts that call turbo run typecheck/test/build.
- `.github/workflows/ci.yml` CI runs pnpm lint, typecheck and test, which go through turbo.

Questions you may get

Q: What does the ^ in turbo.json mean?

It means the same task in the packages this one depends on. ^db:generate runs db:generate in dependencies first. Before web's type check, db's generate runs. Without ^, it refers to a task in the same package.

Tip: A small syntax question that checks you actually wrote the config. Have turbo.json in your head.

Q: What improved after adding Turborepo?

One command checks every package. Forgetting Prisma generate stopped happening. Unchanged packages are skipped thanks to the cache.

Tip: Lead with the concrete problem it solved (generate ordering), not just "it's faster".

Q: Do you use remote caching?

Not now. The cache is local only. In CI, I cache the pnpm store and Docker layers. For a team, I'd add a remote cache. It should cut CI time considerably.

Tip: Honest scope, plus what you'd do on a team. Craftsman uses Turborepo, so expect follow-ups on caching.

Words: monorepo (モノレポ) · task runner (タスクランナー) · task graph (タスクグラフ) · dependency relationship (依存関係) · cache hit (キャッシュヒット) · remote cache (リモートキャッシュ) · workspace (pnpm) (ワークスペース) · parallel execution (並列実行) · regeneration (再生成)

It tests and builds automatically on every push and pull request.

How it works

- CI is a single GitHub Actions workflow.
- It runs on pull requests and on pushes to main.
- The first job runs lint, typecheck and unit tests.
- The same job lints the Helm chart.
- A separate job has Trivy scan dependencies and configuration.
- The web and worker images build in parallel with a matrix.
- The built images are scanned with Trivy too.
- Finally, Docker Compose starts the stack and the E2E tests run.

Why ReelWalk uses it

- The pipeline lives in the same repository as the code.
- Problems are caught at the pull request stage.
- The trade-off is that E2E takes time.
- So E2E runs in parallel with the image builds.

What's real

Real and green: lint, typecheck, unit tests, helm lint, Trivy scans, image builds and Playwright E2E run on every PR. On main the scanned images are pushed to GHCR and a release job commits the new image tag for Argo CD. The OIDC + ECR push steps exist but have never run because no AWS role is configured.

In the repo

- `.github/workflows/ci.yml` Jobs verify, dependency-scan, images (matrix web/worker, GHCR push on main), e2e and release; ECR push gated on vars.AWS_ROLE_ARN.
- `infra/helm/reelwalk/values-gitops.yaml` Image repository and tag. The release job rewrites it with the commit SHA on every merge to main.
- `turbo.json` The typecheck and test tasks that pnpm typecheck / pnpm test fan out across the workspace.
- `e2e/package.json` Script renamed to e2e so turbo run test no longer launched the browser suite in the unit-test job.

Questions you may get

Q: Have you improved CI/CD in any way?

I fixed the unit-test job accidentally running the E2E suite. The e2e package script was named test, so turbo picked it up. Docker layers are cached in the Actions cache. When E2E fails, logs and traces are saved automatically. Trivy's version is pinned in one place.

Tip: Tell one story with cause, fix and result. The turbo test-script collision (commit 1095187) is concrete and easy to explain.

Q: How does CI authenticate to AWS?

No long-lived access keys are stored in GitHub. The design gets a short-lived token via OIDC. That is why the job has the id-token permission. But there is no AWS role yet, so the ECR push has never run. For now, the images go to GHCR instead. It authenticates with the job's own token, so no key is stored there either.

Tip: Be explicit that the ECR push is wired but never executed (skipped until AWS_ROLE_ARN is set), and that the GHCR push is the one that really runs.

Q: What about the CD part?

On a merge to main, CI pushes the images to GHCR. The tag is the commit SHA. Next, CI writes that tag into a values file and commits it. Argo CD detects that commit and applies it to the cluster. CI holds no cluster credentials. But this runs on local kind, not on EKS yet.

Tip: CI and CD are both real now, but only against a local cluster. Say that scope plainly: the registry is GHCR, and ECR and EKS are designed.

Words: workflow (ワークフロー) · job / step (ジョブ / ステップ) · pipeline (パイプライン) · matrix build (マトリックスビルド) · parallel execution (並列実行) · continuous integration (継続的インテグレーション) · cache (キャッシュ) · artifact (成果物) · OpenID Connect (OIDC) · Elastic Container Registry (ECR) · GitHub Container Registry (GHCR)

A GitOps tool that keeps the cluster matching what is in Git.

How it works

- Argo CD is a deployment tool that runs inside the cluster.
- It treats the manifests in Git as the desired state.
- When it finds drift from the cluster, it syncs automatically.
- It is pull-based, rather than CI pushing into the cluster.
- For ReelWalk I wrote the Application manifest.
- It points at the Helm chart and values-kind.yaml.
- prune and selfHeal are enabled.
- CI writes the image tag into Git.
- It is installed and running on local kind.

Why ReelWalk uses it

- Git becomes the single source of truth, with a change history.
- Rolling back is just reverting a commit.
- CI does not need cluster credentials.
- On the other hand, Argo CD itself must be operated.

What's real

Running on local kind only: Argo CD v3.5.3 is installed, the Application auto-syncs main with prune and selfHeal, and a merge to main was seen rolling new GHCR images into the cluster. Never run against EKS or in production.

In the repo

- `infra/argocd/application.yaml` Application tracking main, path `infra/helm/reelwalk` with `values-kind.yaml` and `values-gitops.yaml`, automated prune and selfHeal.
- `infra/argocd/install.sh` Installs Argo CD v3.5.3 into kind with server-side apply, then applies the Application.
- `infra/helm/reelwalk/values-gitops.yaml` Image repository and tag, rewritten by the CI release job on every merge to main.
- `infra/helm/reelwalk/templates/migrate-job.yaml` `argocd.argoproj.io/sync-wave: "-1"` so the migration completes before the Deployments sync.

Questions you may get

Q: What is GitOps?

It makes Git the single source of truth for deployment. The cluster is kept matching the manifests in Git. Manual changes are reverted automatically. Every change is a pull request, so it is reviewed and recorded.

Tip: Contrast push (CI runs kubectl/helm with cluster credentials) and pull (an agent in the cluster watches Git).

Q: Are you actually using Argo CD?

Yes, on local kind. When I merge to main, the new images are deployed automatically. But I have no production experience with it yet. The migration Job has a sync-wave annotation. So the Job finishes before the Deployments sync.

Tip: Scope first (local kind, not production), then one detail (sync-wave -1) that shows you understand ordering.

Q: How do you roll back?

Revert to the previous commit in Git. Argo CD detects it and restores the previous state. But database migrations do not roll back automatically. So migrations must stay backward compatible.

Tip: The database is the hard part of any rollback; mentioning expand-and-contract migrations is a plus.

Words: GitOps (GitOps) · sync (同期) · diff / drift (差分) · self-healing (selfHeal) (自己修復) · prune (プルーン) · declarative (宣言的) · single source of truth (唯一の情報源) · pull-based / push-based deployment (プル型／プッシュ型) · backward compatibility (後方互換)

Testing and security

It scans dependencies, configuration and images for vulnerabilities.

How it works

- Trivy is an open-source security scanner.
- In CI it scans three things.
- First, the dependencies in the pnpm lockfile.
- Second, misconfigurations in the Dockerfiles and Helm chart.
- Third, the built web and worker images.
- A HIGH or CRITICAL finding fails CI.
- Findings with no fixed version are excluded.
- Accepted findings are recorded in a file with the reason.

Why ReelWalk uses it

- Vulnerabilities are caught before deployment.
- It runs from the official image, not a third-party Action.
- That pins the version in one place.
- The trade-off is CI blocking on findings you cannot fix.

What's real

Real: three Trivy scans (lockfile, config, images) gate every CI run at HIGH/CRITICAL; real findings were fixed in commits `be7eb1c` and `c317ca1`, and accepted ones are documented in `.trivyignore.yaml`.

In the repo

- `.github/workflows/ci.yml` TRIVY_IMAGE pinned to 0.75.0; fs, config and image scans with `--exit-code 1`.
- `.trivyignore.yaml` Accepted DS-0002 (root user) findings, each with a written statement: local nginx images, a backup Dockerfile, the GPU splat image.
- `package.json` pnpm overrides for mysql2 and deepmerge-ts, transitive packages pinned by the Prisma CLI.
- `apps/worker/Dockerfile` The fix for the image findings: `--prod` install and package managers removed from the runner.

Questions you may get

Q: How do you handle a vulnerability when one is found?

First I check whether the affected code is actually used. Trivy actually did report HIGH and CRITICAL findings. A stale npm lockfile pinned a vulnerable Next.js. It was unused, so I deleted it. Dependencies pinned by Prisma were raised with pnpm overrides. Then I confirmed CI went green.

Tip: This is a listed job duty. Use the flow: triage (used? fix available?) -> fix or remove -> verify in CI -> record any exception. Commit `be7eb1c`.

Q: What did the image scan find?

There were findings in the bundled npm and the pnpm store. Also old esbuild builds pulled in by test tooling. None of it was used at runtime. So I made the image install production dependencies only. And removed the unused package managers.

Tip: Removing what is not needed beats upgrading it: it shrinks the attack surface. Commit `c317ca1`.

Q: Should every finding be fixed?

No, I decide based on risk. For example, an nginx image was flagged for running as root. But it is only for local Compose and is never deployed. So I accepted it with a written reason. Switching to a non-root image is tracked as follow-up.

Tip: They are checking you do not silently suppress findings: every entry in `.trivyignore.yaml` has a statement, and the scan scope (skipping local-only and unapplied dirs) is commented in `ci.yml`.

Words: vulnerability (脆弱性) · Common Vulnerabilities and Exposures ID (CVE) · severity (深刻度) · misconfiguration (設定ミス) · false positive (誤検知) · accepting (a risk) (許容) · transitive dependency (推移的依存) · supply-chain attack (サプライチェーン攻撃) · fixed version (修正版) · exclusion / ignore (除外)

It checks logic with fast unit tests.

How it works

- Vitest is a fast test runner built on Vite.
- You write tests almost the same way as in Jest.
- In ReelWalk it mainly tests the core package.
- It checks that render job state transitions are correct.
- It tests the decision to process or discard a queue message.
- It also tests which file types can be uploaded.
- The worker handler is tested with the database and S3 mocked.
- CI runs every package through turbo run test.

Why ReelWalk uses it

- Decision logic is split out into pure functions.
- So it can be tested instantly without a database or queue.
- But it cannot check real Chrome or SQS behaviour.
- Playwright E2E covers that.

What's real

Real: Vitest suites in packages/core, packages/render and apps/worker run in CI's verify job on every push.

In the repo

- `packages/core/tests/job-status.test.ts` Allowed transitions: happy path, retries, SUCCEEDED is final, the worker cannot be skipped.
- `packages/core/tests/queue.test.ts` `decideDelivery` (claim, duplicate, stale heartbeat), `decideFailure` exponential backoff, dead-letter handling.
- `packages/core/tests/uploads.test.ts` Upload type resolution and object key rules.
- `apps/worker/tests/handler.test.ts` `handleDelivery` and `handleDeadLetter` with `vi.fn` mocks: no double render, re-queue with backoff, DLQ on last attempt.

Questions you may get

Q: What kind of unit tests did you write?

The most important are the job state transition tests. They check that a succeeded job never leaves that state. A message delivered twice is not rendered twice. They also check that failures retry with exponential backoff.

Tip: Tie it to idempotency: SQS is at-least-once, and the tests prove duplicates are harmless.

Q: What did you do to make the code easy to test?

I separated decisions from side effects. `decideDelivery` only returns what to do. Database updates and S3 happen in the handler. The handler tests mock those with `vi.fn`.

Tip: 'Functional core, imperative shell' is the idea; you don't need the English phrase, just the split.

Q: How do you divide work between unit tests and E2E?

Logic combinations are covered by unit tests. They are fast, so I can write many cases. E2E covers only the important user flows. They are slow, so I keep them few.

Tip: This is the test pyramid; mentioning it by name (テストピラミッド) is fine.

Words: unit test (単体テスト) · test runner (テストランナー) · mock (モック) · state transition (状態遷移) · pure function (純粋関数) · side effect (副作用) · assertion (アサーション) · regression (デグレ) · covering exhaustively (網羅) · exponential backoff (指数バックオフ)

It tests user flows end to end in a real browser.

How it works

- Playwright is a test tool that drives a browser automatically.
- It operates the UI in a real Chromium.
- ReelWalk is for phones, so tests run at a Pixel 7 viewport.
- The main test makes a reel from photos and a video.
- Then it really renders and checks the MP4 can be fetched.
- A corrupt video tests failing after retries.
- In CI, Docker Compose starts every service and the suite runs.
- Locally, parallelism is set to 2.

Why ReelWalk uses it

- Unit tests cannot check how the services connect.
- E2E checks the whole path: web, queue, worker and S3.
- The trade-off is that they are slow and prone to flakiness.
- So CI retries once and keeps traces.

What's real

Real: the suite runs in CI against Docker Compose, including a real render to MP4; it can also point at the kind cluster via E2E_BASE_URL.

In the repo

- `e2e/playwright.config.ts` Pixel 7 project, 5-minute timeout, 2 workers by default (the Windows 10 s stall note), 1 retry and traces on CI.
- `e2e/tests/reels.spec.ts` Create, edit and export a reel; Instagram limits; conflict between tabs; corrupt video fails then retries.
- `e2e/fixtures/corrupt.mp4` A file that passes upload checks but cannot be decoded, to drive the retry and DLQ path.

Questions you may get

Q: What do your E2E tests check?

They run user flows in the browser from start to finish. Making a reel, editing it and exporting it. The render is real: the worker produces an MP4. It checks the file is MP4 and larger than 50 KB. Failure cases use a corrupt video.

Tip: Stress 'no mocks': the E2E goes through SQS (ElasticMQ), the worker, Chrome and S3 (MinIO).

Q: Why two workers locally?

With five in parallel, requests stalled in exact 10-second steps. It happened even straight to MinIO, so it was not the app. It happened between the Windows host and Docker Desktop. Linux CI is unaffected, so only local runs use 2. The root cause is still under investigation.

Tip: A good debugging story: you isolated the layer (原因の切り分け) before blaming the app. Don't claim a root cause you haven't found.

Q: How do you prevent flaky tests?

Wait for state, not for a fixed time. For example, wait until data-status becomes SUCCEEDED. Elements are found by data-testid or role. CI retries once and keeps a trace on failure.

Tip: Retries hide flakiness if overused; one retry plus a trace to investigate is a reasonable balance.

Words: end-to-end test (E2Eテスト) · headless browser (ヘッドレスブラウザ) · locator (ロケーター) · auto-waiting (自動待機) · flaky (フレーキー) · trace (トレース) · parallel execution / number of workers (並列実行／ワーカー数) · viewport (ビューポート) · isolating the cause (troubleshooting) (切り分け)

AWS

A CDN that delivers finished videos quickly from a nearby edge.

How it works

- CloudFront is AWS's CDN.
- It can deliver video fast from edges around the world.
- Today the app returns videos as S3 presigned URLs.
- Setting an environment variable switches to CloudFront URLs.
- In production the design keeps S3 private by using OAC.
- Who can watch is controlled with signed URLs or signed cookies.
- In production OAC is in place; signed URLs are not yet.

Why ReelWalk uses it

- Videos are large, so serving straight from S3 is slower and costlier.
- Each job's output key never changes, so it can be cached for a long time.
- The trade-off is more setup: signing keys, OAC and so on.

What's real

Built: the production distribution (imported into Terraform, origin access control so only CloudFront reads the bucket) serves finished MP4s at `dh5xvp6apljoh.cloudfront.net`; the app uses it when `CLOUDFRONT_BASE_URL` is set and presigned S3 URLs otherwise. No signed URLs: share links are the access control.

In the repo

- `infra/terraform/main.tf` Early `aws_cloudfront_distribution` sketch in front of the media bucket.
- `apps/web/lib/storage.ts` `mediaUrl`: CloudFront when `CLOUDFRONT_BASE_URL` is set, otherwise a presigned GET.
- `docs/adr/0005-local-kubernetes-before-eks.md` Lists CloudFront among things kind does not cover.

Questions you may get

Q: How do finished videos reach the user?

Right now the app returns an S3 presigned URL. Locally MinIO stands in for S3. In production CloudFront delivers the finished videos. The share-link page uses those URLs too.

Tip: CloudFront is real: the distribution was imported into Terraform with origin access control. What is not done is signed URLs; the share link is the access control today.

Q: How do OAC and signed URLs differ?

OAC is CloudFront's permission to read S3. S3 stays private and only CloudFront can read it. A signed URL limits the viewer. It decides who can watch, and until when.

Tip: They protect different hops: OAC is CloudFront to S3, signed URLs are viewer to CloudFront. Production would use both.

Q: How would you refresh the cache?

The output key is built from the job ID. A succeeded job's video never changes. A new export means a new job and a new key. So invalidation is almost never needed.

Words: content delivery network (CDN) · delivery / distribution (配信) · edge location (エッジロケーション) · origin (オリジン) · cache (キャッシュ) · Origin Access Control (OAC) · signed / presigned URL (署名付きURL) · expiry (有効期限) · private (not public) (非公開) · invalidation (無効化)

The plan is AI-drafted captions. It hasn't been built.

How it works

- Bedrock is AWS's generative AI service.
- You can call models such as Claude through an API.
- ReelWalk has no AI features yet.
- What I have in mind is drafting captions.
- It would suggest Instagram text from the listing details.
- I'd make the call from the worker, asynchronously.
- The result is shown as a suggestion and always checked by a person.

Why ReelWalk uses it

- Writing captions is a chore for real-estate agents.
- With Bedrock, access is managed with IAM inside AWS.
- But housing ads have strict wording rules, so review is a must.

What's real

Not built at all: there is no AI code or Bedrock call in the repo, only a one-line 'Bedrock captions are planned' note in docs/ARCHITECTURE.md; nothing beyond this outline is designed.

In the repo

- `docs/ARCHITECTURE.md` Known gaps: 'No AI features yet (Bedrock captions are planned)'.
- `packages/core/src/instagram.ts` Caption limits a generated caption would have to pass.
- `packages/core/src/vibes.ts` Today's caption text comes from data per vibe, with the housing-ad rule.

Questions you may get

Q: Do you already use AI features?

No, not yet implemented. The idea is caption drafts. I lowered its priority and have not tried it yet.

Tip: Answer 'no' cleanly first. An honest 未実装 followed by a sensible plan is far better than vagueness.

Q: If you added it, how would you wire it in?

The same way as rendering: a queue and a worker. Model responses are slow, so the web request doesn't wait. I'd pass only the listing fields so it can't invent facts. The result also goes through the existing Instagram checks.

Q: How do you think about the risks of generative AI?

The biggest is writing wrong facts about a property. Also, housing ads may not express a preference for who buys. So a person always reviews it before use. Cost is controlled with token limits.

Words: generative AI (生成AI) · foundation model (基盤モデル) · prompt (プロンプト) · inference (推論) · hallucination (ハルシネーション) · guardrails (ガードレール) · draft (下書き) · not implemented (未実装) · asynchronous (非同期)

Logs, metrics and alarms show how the system is doing.

How it works

- CloudWatch is AWS's monitoring service.
- Logs, metrics and alarms are in one place.
- Today the worker logs to stdout with the job ID on each line.
- On EKS the plan is to collect that in CloudWatch Logs.
- The key metric is the age of the oldest message in the queue.
- Any message in the DLQ raises an alarm right away.
- I'd also like render time and failure rate as metrics.

Why ReelWalk uses it

- When async work stalls, you don't notice it on screen right away.
- So watching queue backlog and the DLQ gives the most value.
- But more logs and metrics also means more cost.

What's real

Not wired up: web and worker log plain text to stdout and Kubernetes probes check health, but there is no CloudWatch agent, metric or alarm in the repo; everything CloudWatch-specific here is the plan.

In the repo

- `apps/worker/src/handler.ts` Log lines prefixed with [jobId] for each decision and attempt.
- `apps/worker/src/index.ts` /healthz returns 503 if the poll loop stops ticking.
- `infra/helm/reelwalk/templates/worker.yaml` livenessProbe on /healthz; no readiness probe for a queue consumer.

Questions you may get

Q: What would you monitor in production?

First, the age of the oldest message in the queue. If it grows, workers are too few or stuck. Next, DLQ message count. Even one is an alarm. Then render failure rate and duration.

Tip: `ApproximateAgeOfOldestMessage` beats queue length: it shows user-facing delay directly.

Q: How would you notice a stuck worker?

The worker has a health check endpoint. If the loop stops for a while, it returns 503. Then Kubernetes restarts it. The job is taken over by another pod after the timeout.

Tip: This part is real and runs on kind. Separate it clearly from the CloudWatch alarms, which are only planned.

Q: How would you improve the logs?

They are plain text now; I'd move to structured JSON. Job ID, generation and attempt become fields. Then CloudWatch can search and aggregate them.

Words: monitoring (監視) · metrics (メトリクス) · alarm (アラーム) · threshold (閾値) · backlog / messages piling up (滞留) · structured logging (構造化ログ) · observability (可観測性) · standard output (stdout) (標準出力) · health / liveness monitoring (死活監視)

Each workload gets only the permissions it needs, through a role.

How it works

- AWS permissions are granted with IAM roles.
- The rule is: no access keys inside containers.
- On EKS, Pod Identity or IRSA attaches a role to a pod.
- The web app only sends messages; the worker only receives and deletes.
- That is the principle of least privilege.
- The database password is passed in a Kubernetes Secret.
- In production that Secret is managed outside the chart.
- From CI to AWS, OIDC provides temporary credentials.

Why ReelWalk uses it

- Long-lived keys do a lot of damage if they leak.
- With roles, credentials rotate automatically.
- However, the chart currently has a single ServiceAccount.
- Splitting it between web and worker is the next improvement.

What's real

Built: CI assumes an IAM role through GitHub OIDC to push to ECR (no stored keys); on EKS pods used Pod Identity; on App Runner and Fargate the instance and task roles carry the same scoped policy; credentials sit in SSM Parameter Store. Locally the Secret holds throwaway MinIO and Postgres credentials.

In the repo

- `infra/helm/reelwalk/values.yaml` serviceAccount annotations for IRSA and secret.create / existingSecret.
- `infra/helm/reelwalk/templates/config.yaml` The ConfigMap, the optional Secret and the shared ServiceAccount.
- `.github/workflows/ci.yml` id-token: write and configure-aws-credentials with a role, skipped until configured.
- `docs/adr/0004-kubernetes-and-helm.md` ConfigMap vs Secret, and IRSA in production.

Questions you may get

Q: What permissions would the worker get?

Only receive, delete and change-visibility on the render queue. S3 is limited to reading and writing one bucket. Resources are named by ARN; wildcards are avoided. The web app gets only send and presigned-URL signing.

Tip: Concrete actions (`sqs:ReceiveMessage`, `sqs:DeleteMessage`, `sqs:ChangeMessageVisibility`) show you have thought it through. This policy is live: it is attached to the Fargate task role and the App Runner instance role.

Q: How do you manage secrets?

On local kind, throwaway values go into a Secret. In production the design is that the chart doesn't create it. It would be synced from Secrets Manager with External Secrets. No secrets ever go into Git.

Q: What's the difference between IRSA and Pod Identity?

Both give a pod an IAM role. IRSA uses an OIDC provider and an annotation on the SA. Pod Identity is newer and simpler to set up. For a new cluster I'd choose Pod Identity.

Tip: The chart's comment mentions IRSA; either works. Saying you'd pick Pod Identity for new clusters shows current knowledge.

Words: least privilege (最小権限) · IAM role (IAMロール) · policy (ポリシー) · credentials (認証情報) · secret (シークレット) · service account (Kubernetes) (サービスアカウント) · temporary credentials (一時的な認証情報) · leak (漏洩) · rotation (ローテーション) · trust policy (信頼ポリシー)

AWS's managed Kubernetes service.

How it works

- EKS stands for Amazon Elastic Kubernetes Service.
- AWS runs the control plane and makes it redundant.
- We can focus on the nodes and the app.
- In ReelWalk, I ran the same Helm chart on EKS for a day.
- Pods got AWS permissions through Pod Identity.
- But most of the \$250 a month was idle cost, so I retired it.
- No static access keys.

Why ReelWalk uses it

- Running the control plane yourself is hard work.
- With EKS, availability and upgrades are handled by AWS.
- On the other hand, the control plane alone costs money.
- So the order is: verify on kind first, then move.

What's real

Built and retired: an EKS 1.34 cluster (Terraform, one t3.xlarge node, Pod Identity, RDS, ingress-nginx, cert-manager) ran reelwalking.com for one day on 2026-10-07 with a real render verified end to end, then was destroyed because it cost about \$250 a month for an idle app (ADR 0014, superseded by ADR 0015).

In the repo

- `docs/adr/0005-local-kubernetes-before-eks.md` Why kind before EKS, and what only a real EKS cluster can prove.
- `infra/helm/reelwalk/values.yaml` Defaults for a real cluster: empty AWS endpoints, external Secret, the serviceAccount annotation slot for IRSA.

Questions you may get

Q: Do you have experience running EKS?

To be honest, I have not run EKS in production yet. My AWS account verification never completed, so I could not deploy. Instead, I verified the same chart on a local kind cluster. The EKS-specific differences are in values.yaml and the IRSA design. After joining I want to catch up quickly on real clusters.

Tip: Do not bluff: the interviewer can probe node groups, upgrades and IRSA. An honest answer with a concrete plan scores better.

Q: How do pods get AWS permissions?

The design uses IRSA. The ServiceAccount is annotated with an IAM role ARN. Pods automatically receive short-lived credentials. There is no need to put access keys into a Secret. values.yaml already has a slot for that.

Tip: If they push further, mention EKS Pod Identity as the newer option, and least-privilege policies per workload.

Q: What would moving to EKS take?

First, create the VPC, EKS, RDS and so on with Terraform. Then prepare a values file for EKS. CI pushes the images to ECR. The chart and application code should not need changes. But that is an untested hypothesis.

Tip: ADR 0005 says exactly this: 'That claim is untested until it is done.'

Words: managed service (マネージドサービス) · control plane (コントロールプレーン) · worker nodes / node group (ワーカーノード / ノードグループ) · IAM Roles for Service Accounts (IRSA) · ServiceAccount (サービスアカウント) · redundancy (making something redundant) (冗長化) · availability (可用性) · running / in operation (稼働) · load balancer (ロードバランサー)

It defines and creates AWS resources as code.

How it works

- Terraform is an IaC tool: infrastructure written as code.
- You declare the desired state in a language called HCL.
- plan shows the diff and apply makes the change.
- ReelWalk defines S3, SQS and CloudFront.
- RDS and ECS are written as placeholders too.
- But it has never been applied.
- It is an early draft, so it differs from the current design.

Why ReelWalk uses it

- Manual setup can be neither reproduced nor reviewed.
- As code, changes are reviewed in pull requests.
- On the other hand, the state file must be managed safely.

What's real

Applied and live: remote state in S3 with lock files; the media bucket and CloudFront (imported), SQS with a DLQ, ECR, the GitHub OIDC role, the app IAM policy, a budget, the VPC, App Runner, the Fargate task definition and SSM parameters. EKS and RDS were created and destroyed the same day. CI scans infra/terraform with Trivy.

In the repo

- `infra/terraform/main.tf` S3 bucket, SQS queue (no DLQ yet), CloudFront in front of S3, and ECS/RDS/Redis placeholders.
- `infra/terraform/variables.tf` Region, project and bucket name variables.
- `docs/adr/0004-kubernetes-and-helm.md` Notes that the Terraform started as an ECS/Fargate design before Kubernetes was chosen.

Questions you may get

Q: What do you manage with Terraform?

The S3 bucket and CloudFront for media, the SQS queues, ECR, and the IAM role for GitHub. Also the App Runner service, the Fargate task definition and the SSM parameters. State lives in S3; I check the plan diff before every apply. I also created EKS and RDS once, deleted them over cost, and recorded it in an ADR.

Tip: It is live. Existing resources were imported, not recreated, so the CloudFront domain survived. The scanner findings (no WAF, no customer-managed key, public subnets) are accepted with reasons in `.trivyignore.yaml`.

Q: What are the problems with the current Terraform?

It was written early, so it no longer matches the architecture. It has ECS and Redis, but we now use Kubernetes and no Redis. The SQS DLQ is not defined yet either. State is planned to live in S3 with locking. EKS and VPC definitions still need to be added.

Tip: Self-critique is well received in Japanese interviews; it shows you know what 'done' looks like.

Q: How do you split responsibilities between Terraform and Helm?

Terraform builds the AWS foundation. VPC, EKS, RDS, S3, SQS and so on. Helm deploys the app on top of it. They change at different rates, so they are managed separately.

Tip: Lifecycle separation is the key idea: infrastructure changes rarely, the app changes every release.

Words: Infrastructure as Code (IaC) · provider (プロバイダー) · resource (リソース) · state file (state (ステート)) · plan / apply (plan / apply) · diff (差分) · drift (real infra differs from code) (ドリフト) · reproducibility (再現性) · definition (定義)

8. What's real, what's designed

Never claim the designed column as done. Say "designed and verified locally" instead.

Component	Status	Detail
TypeScript + React	Built and running	Built and used everywhere: TypeScript across apps and packages, React 19 in the web app, zod 4 schemas in packages/core and in every Server Action.
Next.js	Built and running	Built with Next.js 16; runs locally (dev server, Docker Compose, kind) and in production on AWS App Runner at reelwalking.com (ADR 0015).
Server Components	Built and running	Built: the reels grid, editor page, export screen and exports list are async Server Components reading Postgres through Prisma; running locally and in production.
Server Actions	Built and running	Built: createReel, saveReel (revision check), saveCaption, createUpload, confirmUpload, exportReel and retryRenderJob are real; there is no login yet, so the workspace comes from a seeded demo user.
Remotion	Built and running	Built: the Player preview and worker renders to MP4 both work locally and in the Playwright e2e run; ADR 0006 records the decision.
Prisma	Built and running	Built: Prisma 7 with @prisma/adapters-pg, seven committed migrations, a migrate Job in the Helm chart (kind, and EKS for a day), and a one-off Fargate task that applies them in production.
PostgreSQL	Built and running	Built: PostgreSQL 16 runs locally in Docker Compose and kind; production used RDS for a day (ADR 0014) and now uses the free Postgres tier at Supabase (ADR 0015). The conditional updates are real and tested.
S3	Runs locally with a stand-in	The code uses the AWS SDK v3 S3 client and presigned URLs, but it has only run against MinIO (Docker Compose and kind); the S3 bucket and CloudFront exist only in the Terraform sketch.
SQS	Runs locally with a stand-in	The code uses the real AWS SDK SQS client against ElasticMQ locally (Docker Compose and kind) and against SQS in production, where Terraform creates the queue and its dead-letter queue with the same lease and receive count (ADR 0011, 0015).
Render worker	Runs locally with a stand-in	Built and tested (Vitest handler tests, a real render in Playwright); it runs in Docker Compose and as 2 pods on kind against ElasticMQ and MinIO, never on EKS. In production the real AWS service is used (ADR 0011, 0015).
Job state machine	Built and running	Fully built: the transition table and its tests live in packages/core, and every status write in the worker and the Server Actions is a conditional Prisma updateMany on Postgres.
Idempotency	Built and running	Built and covered by Vitest handler tests (duplicate delivery, stale generation, take-over of a dead worker's job); exercised locally with ElasticMQ, never under production load. Per-claim fencing tokens are not implemented. The transactional outbox is built (ADR 0010) and tested against a real Postgres.
Retries & DLQ	Runs locally with a stand-in	Backoff retries, the 3-receive redrive and the DLQ consumer are built and tested, and run against ElasticMQ's dead-letter queue locally; the real SQS redrive policy is not in Terraform yet. In production the real AWS service is used (ADR 0011, 0015).
Docker	Built and running	Real: the web and worker images build in CI on every push; on main they are pushed to GHCR and to ECR (GitHub OIDC role, no stored keys). App Runner and Fargate run the ECR images in production.
Kubernetes	Built and running	The Deployments, Service, probes, resources and migration Job are real: they run on a local kind cluster and ran on EKS for one day (ADR 0014) before production moved to App Runner and Fargate for cost (ADR 0015).

Helm	Built and running	Real: the chart is linted in CI and installed on the local kind cluster with values-kind.yaml; values.yaml (the AWS defaults) has never been installed anywhere.
kind	Built and running	Real: infra/kind/up.sh creates the cluster, loads the images and installs the chart locally; kind is the stand-in for EKS, which was never created.
Turborepo	Built and running	Built: `pnpm typecheck/test/build` run through turbo locally and in GitHub Actions; caching is local only (no remote cache), and the Dockerfiles do not use turbo prune.
GitHub Actions	Built and running	Real and green: lint, typecheck, unit tests, helm lint, Trivy scans, image builds and Playwright E2E run on every PR. On main the scanned images are pushed to GHCR and a release job commits the new image tag for Argo CD. The OIDC + ECR push steps exist but have never run because no AWS role is configured.
Argo CD	Runs locally with a stand-in	Running on local kind only: Argo CD v3.5.3 is installed, the Application auto-syncs main with prune and selfHeal, and a merge to main was seen rolling new GHCR images into the cluster. Never run against EKS or in production.
Trivy	Built and running	Real: three Trivy scans (lockfile, config, images) gate every CI run at HIGH/CRITICAL; real findings were fixed in commits be7eb1c and c317ca1, and accepted ones are documented in .trivyignore.yaml.
Vitest	Built and running	Real: Vitest suites in packages/core, packages/render and apps/worker run in CI's verify job on every push.
Playwright	Built and running	Real: the suite runs in CI against Docker Compose, including a real render to MP4; it can also point at the kind cluster via E2E_BASE_URL.
CloudFront	Built and running	Built: the production distribution (imported into Terraform, origin access control so only CloudFront reads the bucket) serves finished MP4s at dh5xvp6apljoh.cloudfront.net; the app uses it when CLOUDFRONT_BASE_URL is set and presigned S3 URLs otherwise. No signed URLs: share links are the access control.
Bedrock	Designed, not deployed	Not built at all: there is no AI code or Bedrock call in the repo, only a one-line 'Bedrock captions are planned' note in docs/ARCHITECTURE.md; nothing beyond this outline is designed.
CloudWatch	Designed, not deployed	Not wired up: web and worker log plain text to stdout and Kubernetes probes check health, but there is no CloudWatch agent, metric or alarm in the repo; everything CloudWatch-specific here is the plan.
IAM & Secrets	Built and running	Built: CI assumes an IAM role through GitHub OIDC to push to ECR (no stored keys); on EKS pods used Pod Identity; on App Runner and Fargate the instance and task roles carry the same scoped policy; credentials sit in SSM Parameter Store. Locally the Secret holds throwaway MinIO and Postgres credentials.
EKS	Built and running	Built and retired: an EKS 1.34 cluster (Terraform, one t3.xlarge node, Pod Identity, RDS, ingress-nginx, cert-manager) ran reelwalking.com for one day on 2026-10-07 with a real render verified end to end, then was destroyed because it cost about \$250 a month for an idle app (ADR 0014, superseded by ADR 0015).
Terraform	Built and running	Applied and live: remote state in S3 with lock files; the media bucket and CloudFront (imported), SQS with a DLQ, ECR, the GitHub OIDC role, the app IAM policy, a budget, the VPC, App Runner, the Fargate task definition and SSM parameters. EKS and RDS were created and destroyed the same day. CI scans infra/terraform with Trivy.

9. General interview questions

Q: What are your strengths?

My strength is understanding how things work before I build. In ReelWalk, I recorded why I use each technology in ADRs. For example, why I chose SQS and which options I rejected. So I can explain the reasoning behind a decision even later on.

Tip: One strength, one piece of evidence they can check in the repo. Don't list three adjectives.

Q: What is your weakness?

I sometimes dig too deep on my own. In ReelWalk, I spent too long investigating a delay that only happened locally. Now I set a time box and share progress partway through. In a team, asking early is faster overall.

Tip: A real weakness plus what you now do about it. The 10 s stall is true: Playwright traces, then a 2-worker workaround instead of chasing it forever.

Q: Tell me about your experience developing in a team.

At my previous job, I worked on 【what】 in a team of 【number】. We worked with 【process, e.g. a daily stand-up and two-week sprints】. What I paid attention to was making my progress visible early. 【One concrete episode】

Tip: Their team is a PO plus three developers, works from documents and tickets, and checks the day's plan every morning. Mirror that if it is true for you.

Q: What do you pay attention to in code review?

First, I check the change does what the description says. Next, I look at behaviour on errors and under concurrency. When I comment, I always give the reason and an alternative. Matters of taste I mark as "nit" so the author can decide. As an author, I keep pull requests small.

Tip: The job post asks for someone who considers the other person (相手を慮る). Show you review the code, not the person.

Q: How do you choose technologies?

First I pin down the problem and the constraints. Then I narrow it to two or three options and compare trade-offs. For the queue, for example, I compared Redis, PostgreSQL and SQS. I chose SQS because redelivery and a dead-letter queue come built in. I record the reasons as ADRs.

Tip: At this company technology choices are the team's call, so they want people who can justify one. Mention a rejected option.

Q: Tell me about a failure and what you learned.

In the first version, I passed jobs through a Redis list. When a worker crashed mid-job, the job was lost. The cause was that I'd assumed the queue would never fail. Since then I design assuming failures and duplicates will happen. Now it's handled with SQS and idempotent processing.

Tip: Own the mistake in one line, spend the rest on the lesson. If you have a work example, use it instead and keep this one for the ReelWalk questions.

Q: What technology are you learning lately?

Lately I've been focusing on Kubernetes. I ran a Helm chart on kind and set up probes and resource limits. I'm aiming for CKAD level, studying with 【method】. I've also experimented with Gaussian splatting for 3D property video.

Tip: CKAD-level Kubernetes is a plus in the post. Name something you actually configured (readiness vs liveness is a good follow-up to be ready for).

Q: How do you handle a vulnerability?

First I check which dependency is affected and where it's used. Then I set the priority from severity and whether it's actually exploitable. If there's a fix, I update and confirm with tests that nothing broke. If it can't be fixed right away, I accept it temporarily with a written reason. In ReelWalk, every Trivy exception has its reason written down. I've set up Dependabot to propose dependency updates weekly.

Tip: This is a listed duty. Show a process (scope, exploitability, fix, verify, record), not just "I run npm audit".

Q: Is there anything you'd like to improve in CI/CD?

First, the speed of feedback. Turborepo's cache can skip packages that haven't changed. Second, moving security checks earlier in the pipeline. Third, using Argo CD to keep production in line with what's in Git. But I'd want to hear your current pain points before setting priorities.

Tip: CI/CD improvement is a listed duty. Offer ideas, then defer to their reality; proposing fixes before you know the pipeline sounds arrogant.

Q: What is your approach to testing?

I pull logic out into pure functions and test them with Vitest. For example, job state transitions and retry decisions. User flows are checked in a real browser with Playwright. The end-to-end tests go as far as actually rendering a video. End-to-end tests are slow, so most of the tests are unit tests.

Tip: Their stack is Vitest plus Playwright, the same split. Mention the test pyramid without using jargon for its own sake.

Q: Why do you want to work in Japan?

I became interested in Japan through 【your connection, e.g. study or work】. I respect how carefully Japanese teams approach quality. I felt the same in your phrase "build software with integrity." 【Your situation or plans for living in Japan】 I want to work in Japan for the long term and grow as an engineer.

Tip: They want to know you will stay. Be concrete about visa, location and timeline if asked; keep it positive about Japan, not negative about home.

Q: Are you worried about working in Japanese?

I can handle technical conversations if they're not too fast. My reading and writing are at JLPT N2 level. When I don't know a word, I check it on the spot. I find important decisions are safest written down in tickets or chat. Every day I improve my work Japanese through 【method】.

Tip: This whole interview is the answer. Be honest about the level, show a strategy; their team works from documents and tickets, which helps you.

Q: What would you like to do after joining?

First, I want to understand the shipping tool's code and its business. I'd start with small changes and bug fixes and ship something early. After that I'd like to work on improving CI/CD and vulnerability handling. Eventually I want to be trusted with both the app and the infrastructure.

Tip: Show a ramp: learn, ship small, then improve. Naming their product shows you read the post.

Q: How do you use AI tools?

I use AI coding agents every day. I built ReelWalk together with Claude Code too. But I make the design decisions myself and record the reasons in ADRs. I always check generated code with tests and review. I also follow rules on usage, like never pasting secrets.

Tip: The company promotes AI use, and the public repo shows AI co-authored commits, so be open about it. The point to land: you own the decisions and can explain every part.

Q: What do you do when the team disagrees?

First I ask about the reasoning behind the other view. Usually it's just different assumptions or priorities. Once we agree on criteria, we compare with a small experiment. Once it's decided, I commit fully even if it wasn't my idea.

Tip: Value: チームで価値を最大化する. They are checking cooperation, not who wins.

Q: Why are you looking for a new job?

【One positive line about your current or last job】 Along the way, I came to want to work on both the app and the infrastructure. Building ReelWalk, I found Kubernetes and CI/CD especially interesting. I want to apply that on a real product, with a real team.

Tip: Never criticise a previous employer. Frame it as moving toward something they offer.

Q: Are you stronger in frontend or backend?

In terms of experience, I've done more 【frontend / backend】. But in ReelWalk I built both myself. Autosave, for example, spans both the screen and the server. Saves based on an old revision are rejected, so a second tab can't overwrite the first. More than either side, I enjoy designing the boundary between them.

Tip: Full-stack roles want someone comfortable at the seam. The revision check (ADR 0006) is a good concrete example.

Q: How do you respond to a production incident?

First I check the impact and tell the people affected. Then I stop the user impact with a quick mitigation. Once things are stable, I find the root cause from the logs. Finally we agree on prevention and write up a retrospective. 【One line from a real incident, if you have one】

Tip: Order matters: communicate, mitigate, then investigate. Don't claim on-call experience you don't have.

Q: Is working in the office OK for you?

Yes, no problem. 【Where you live and the commute to Kanda】 Sitting close makes it easy to ask questions, so I think it suits me.

Tip: The post asks for office work at the Kanda office, and says the team sits close and talks as needed.

About Craftsman Software

- Their message is "Toward a world where we can focus more on creating." They support software end to end, from development through infrastructure to operations.
- Their philosophy is "Build software with integrity": take responsibility for design, implementation and operation alike.
- Their four values: face users with integrity, solve problems with technology, maximise value as a team, build the future with automation. Technology choices are left to each team, and anyone can speak up regardless of seniority.
- Their own product, AppThrust, builds a Kubernetes-based application platform in the customer's own AWS account. A push to GitHub triggers a build, and a new revision can be released gradually, starting with 1% of traffic.
- In September 2026 they exhibited AppThrust at Platform Engineering Kaigi 2026 and Tokyo Game Show 2026, and they are exhibiting next at CEATEC 2026 (13-16 October).
- The team you would join maintains and extends a shipping-management tool built with Next.js Server Components and Server Actions. It launched with a minimal feature set in April 2024; the team is one product owner and three developers, working in an agile way.
- Nearly 90% of staff are engineers, including a top CNCF contributor. Director Hidehito Nozawa is one of the authors of "Survival TypeScript".
- Study sessions happen during working hours, and contributing to work-related OSS is allowed. The company actively promotes using AI, overtime averages about 5 hours a month, and they are considering expanding to the US and hiring globally.

Questions to ask them

- What is the shipping-management team focusing on most right now?
- How do you currently run CI/CD and handle vulnerabilities?
- I'd like to hear about anything that was hard once you ran Server Actions and Prisma in production.
- Does what you learn building AppThrust feed back into client projects?
- What would you expect me to be able to do after my first three months?
- How does code review work on the team, and what do reviewers look for?
- I read that the company promotes using AI. How does the team use it day to day?
- Have you worked with team members whose first language is not Japanese?